

Подготовка к демонстрационному экзамену.

Методическое пособие по решению типового задания.

Содержание

Текст задания.....	3
Подготовка рабочего пространства.....	5
Подключение сервера	5
Работа с архивом фреймворка	6
Настройка фреймворка и работа с системой контроля версий (git).....	7
Создание БД.....	13
Авторизация пользователя.....	18
Регистрация и разработка интерфейса.....	23
Создание и обработка заявок.....	35
Создание правил для заявок.....	35
Создание функционала заявок пользователя	41
Формирование ответов на заявки администратором.....	50
Дополнительное улучшение интерфейсов	54
Пользовательские карточки	54

Текст задания

Текст задания и формулировки сохранены из типового задания.

Разработать информационную систему для соответствующей предметной области.

Инструкция к выполнению практической части:

Необходимо разработать базу данных, позволяющую реализовать функционал разрабатываемой информационной системы с учетом особенностей предметной области. В качестве документации к системе разработайте диаграмму базы данных с помощью средств графических редакторов. Используйте все знания в области языков разметки и программирования, но не забывайте что избыточность кода-моветон, а кодовое разнообразие – признак хорошего вкуса. Каждый раздел системы должен иметь свою отличительную черту. Для удобства разработки можете воспользоваться фреймворком. Не перегружайте вашу разработку обилием цветов, оставьте это для живописцев. При разработке поставьте себя на место потенциального клиента и постарайтесь создать идеальную информационную систему, как с точки зрения пользователя, так и с точки зрения разработчика. Помните, что пользователи разные. Так же как и устройства, с которых они осуществляют вход в Интернет.

Вам необходимо также разработать дизайн всех страниц для использования со смартфоном с разрешением 390x844 px. Дизайн можно представить в виде файлов изображений .png (отдельное изображение для каждой страницы), либо в виде .html файлов (отдельный файл для каждой страницы).

Интегрируйте Ваш дизайн в разрабатываемую информационную систему. Предусмотрите анимацию для улучшения пользовательского опыта.

Описание предметной области:

Портал клининговых услуг «Мой Не Сам» представляет собой информационную систему для формирования заявок на уборку жилых и производственных помещений. Перед началом использования портала пользователю необходимо пройти процедуру регистрации. В ходе регистрации он указывает данные о себе (ФИО, телефон, адрес электронной почты, логин и пароль) (логины разных пользователей не должны совпадать).

Войдя в систему, заказчик может сформировать заявку на получение услуг, указав свой адрес, контактные данные и конкретный вид услуги из списка предоставляемых. Также заказчик может указать желаемую дату и время получения услуги, а также предпочтительный тип оплаты (наличными или с помощью банковской карты).

Заявки заказчиков хранятся в системе. В каждой заявке находятся данные заказчика, статус оказания услуги (новая заявка, услуга оказана или услуга отменена).

После подачи заявки администратор может подтвердить заявку, поставить отметку о выполнении или отклонить заявку с указанием причины отклонения.

Основной функционал информационной системы:

1. Страница регистрации. На данной странице необходимо предусмотреть добавление пользователя в систему. Пользователю необходимо предоставить возможность ввести уникальный логин, пароль (минимум 6 символов), ФИО (символы кириллицы и пробелы), телефон (в формате +7(XXX)-XXX-XX-XX), адрес электронной почты (формат электронной почты). Все поля обязательны для заполнения. Ошибки валидации должны

отображаться на форме. По кнопке «Зарегистрироваться» пользователь должен заноситься в базу, если поля прошли валидацию.

2. Страница авторизации. На данной странице необходимо предусмотреть возможность ввода логина и пароля для зарегистрированных пользователей. Попытки некорректного ввода логина и пароля должны сопровождаться сообщениями.

3. Страница создания заявки. На данной странице авторизованный пользователь имеет возможность просмотреть историю своих заявок, а также оставить новую заявку.

4. Страница формирования заявки. Пользователь указывает: свой адрес, контактные данные (номер телефона в формате +7(XXX)-XXX-XX-XX), желаемую дату и время получения услуги. Помимо этого, пользователь выбирает вид услуги из предоставленного списка (общий клининг, генеральная уборка, послестроительная уборка, химчистка ковров и мебели). Если требуемый вид услуги отсутствует в списке, пользователь отмечает соответствующий чекбокс («Иная услуга»), после чего ему становится доступно поле для ввода текстовой информации, где он описывает, какая конкретно услуга ему необходима. Также пользователь указывает предпочтительный тип оплаты (наличные или банковская карта). Все поля обязательны для заполнения.

5. Панель администратора. Доступ в панель администратора осуществляется по логину adminka и паролю password. В панели администратора видны все заявки (ФИО заявителя, контактные данные и иные свойства заявки). Администратор может сменить статус на «в работе», «выполнено» или «отменено» (обязательно с указанием причины отмены).

Подготовка рабочего пространства

Подключение сервера

У нас нет синхронизации с удаленным сервером, поэтому первым этапом необходимо создать такое подключение, для этого:

1. Заходим в **личный кабинет** (ЛК) участника (по типовому адресу wsr.ru) и авторизируемся через данные, которые у вас предоставлены в карточке;
2. Создаём локальную папку на своём ПК, для этого открываем терминал (win+T) и вводим команду (пример из ЛК):

```
mkdir server
```

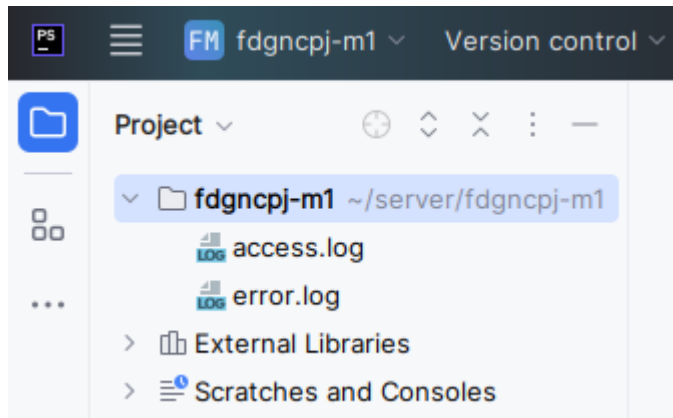
3. При первом подключении введите вторую команду из ЛК, затем введите «yes», чтобы сделать узел доверенным, потом вводим пароль (**он никак не отображается и это нормально**):

```
sshfs zxrysnch@192.168.14.26:/srv/users/zxrysnch ~/server/
```

4. На случай перезагрузки или вылета профиля, а также если вы уже настраивали узел как доверенный, то можно скопировать третью команду из ЛК, чтобы не вводить пароль:

```
echo a33fBm | sshfs -o password_stdin zxrysnch@192.168.14.26:/srv/users/zxrysnch ~/server/
```

5. Открыть в **PhpStorm** необходимый модуль как проект (в данном примере xxx-m1):



Работа с архивом фреймворка

Теперь необходимо развернуть фреймворк в нашей папке с проектом и настроить его для работы, действуем в таком порядке:

Копируем архив с фреймворком из папки «public» в наш модуль, например сейчас в «fdgncpj-m1». Можно это сделать «классическим» способом через проводник операционной системы и распаковать, либо ввести команды в терминал (в будущем они нам пригодятся):

1. Для начала подключимся к серверу по ssh, вводим в терминал команду для подключения (не забудьте поменять логин на свой, а не со скриншота!):

```
ssh zxrysneh@192.168.14.26
```

2. Вводим пароль (**напомним, что его не видно и это нормально**)

3. По навигации и переходом по каталогам сервера вам необходимо использовать две команды: **ls** – список всех файлов и папок из текущего каталога, **cd** – смена каталога. Например, чтобы перейти в каталог m1 нужно ввести команды:

```
zxrysneh@wsr:~$ ls
fdgncpj-m1 fdgncpj-m2 fdgncpj-m3 fdgncpj-m4 fdgncpj-m5 public
zxrysneh@wsr:~$ cd fdgncpj-m1
```

4. Теперь копируем через терминал архив к нам в каталог (**название и формат архива могут отличаться**). Обратите внимание на формат команды копирования, который принимает два параметра (откуда и куда), поэтому мы использовали **две точки**, чтобы показать, где относительно нашего каталога лежит архив (на пути выше, чтобы «зайти» в «public»), а также нам необходимо копировать наш архив в корень нашей папки (для этого вторым параметром мы ставим **точку**):

```
zxrysneh@wsr:~/fdgncpj-m1$ cp ../public/yii-basic-app-2.0.49.tar .
```

5. Теперь распакуем архив, но тут всё зависит его формата. Если у нас **zip** архив, то распакуем его командой **unzip yii2-archive.zip**, если это архив **tgz**, то команда будет выглядеть примерно так **tar -xzf yii2-archive.tgz**, в нашем же случае мы будем распаковывать архив так:

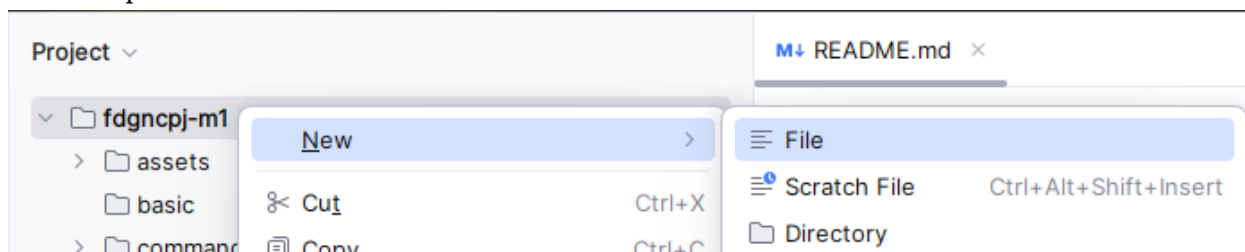
```
zxrysneh@wsr:~/fdgncpj-m1$ tar -xvf yii-basic-app-2.0.49.tar
```

6. После распаковки наши файлы фреймворка находятся в папке basic (в нашем случае), но их необходимо поместить в корень нашей папки. «Классически» мы можем зайти через проводник в каталог «basic» выделить все каталоги, файлы и вырезать их в корень нашего проекта. **Внимание!!! Не забудьте включить отображение скрытых файлов в системе, чтобы ничего не потерять.** Горячие клавиши **Ctrl+N** или вкладка **Вид -> Показать скрытые** (в проводнике системы). Также можно перенести все файлы командой (кроме, скрытых которые мы вырежем и перенесём сами), принцип такой же, как и при копировании (обратите внимание, что опять второй параметр будет **точка**, так как нужно переносить в корень):

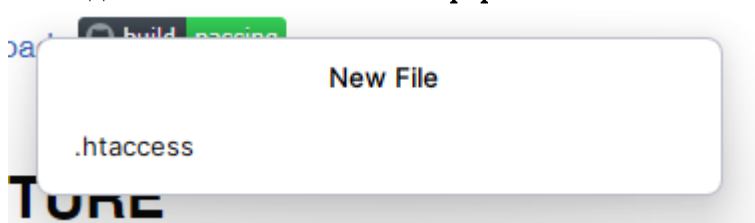
```
zxrysneh@wsr:~/fdgncpj-m1$ mv basic/* .
```

Настройка фреймворка и работа с системой контроля версий (git)

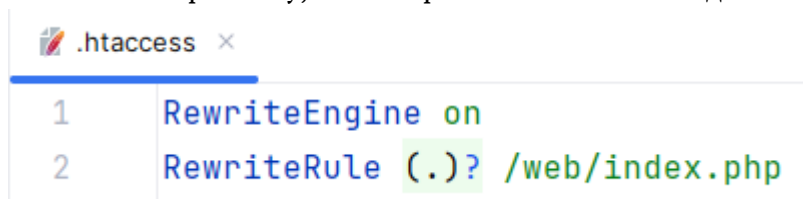
Настроим все необходимые параметры для работы с проектом. Для начала сделаем так, чтобы наш проект открывался сразу в браузере из корня нашего проекта (по умолчанию он отрывается структурой и нам нужно нажать на папку «web» в браузере). Создадим правило переадресации на «web». Создаем через проводник IDE **PhpStorm** новый файл в **корне** нашего проекта.



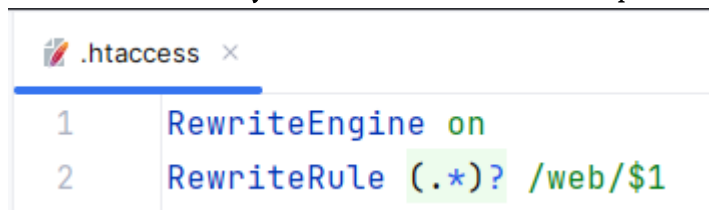
Файл должен быть **без имени** и **формата htaccess**



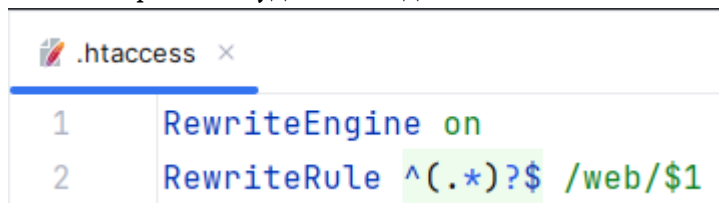
Форматы переадресации пишутся через регулярные выражение (RegEx), поэтому их можно написать по-разному, самой простой способ выглядит так:



Но **минимально лучше** использовать такое правило:



Полное правило будет выглядеть так:

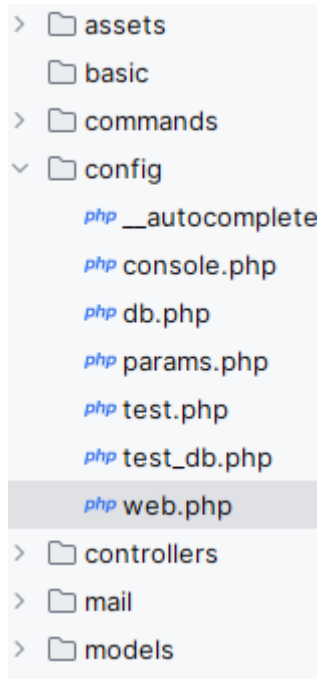


После этого можете открыть в браузере из ЛК наш проект. Если у вас сразу открылся интерфейс проекта, то пропустите следующий шаг. Если у вас появилась ошибка такого вида (**Внимание!!! Другие ошибки чаще всего говорят, что потеряны файлы, а это значит необходимо удалить всё и распаковать архив заново**):

Invalid Configuration – yii\base\InvalidConfigException

yii\web\Request::cookieValidationKey must be configured with a secret key.

В проводнике PhpStorm открываем файл «web.php» в папке «config»



Находим ключ «cookieValidationKey» внутри «components» и в пустые кавычки записываем случайную строку, например:

```
13         ],
14         'components' => [
15             'request' => [
16                 // !!! insert a secret key in the following (if it
17                 'cookieValidationKey' => 'superSecretKey',
18             ],
19             'cache' => [
20                 'class' => 'yii\caching\FileCache',
21             ],
```

Обновите страницу в браузере с вашим проектом и появиться стандартный интерфейс фреймворка.

Осталось настроить ещё несколько параметров, чтобы начать разработку. Первым делом сделаем чтобы наши пути в адресной строке браузера стали более удобными, для этого раскомментируем параметр «urlManager» в том же файле «web.php»

```
44         'db' => $db,
45         'urlManager' => [
46             'enablePrettyUrl' => true,
47             'showScriptName' => false,
48             'rules' => [
49             ],
50         ],
51     ],
52     'params' => $params,
```

Ниже сразу можно раскомментировать ещё две строки заменив параметры доступа, одна из которых даст нам доступ к «панели отладки» фреймворка (ip адрес берём из ЛК «YOUR IP»), а вторая доступ к генераторам (она нам обязательно нужна, её тоже можно настроить по-нашему ip, так как при проверке она не учитывается):

```
55     if (YII_ENV_DEV) {
56         // configuration adjustments for 'dev' en
57         $config['bootstrap'][] = 'debug';
58         $config['modules']['debug'] = [
59             'class' => 'yii\debug\Module',
60             // uncomment the following to add you
61             'allowedIPs' => ['192.168.13.234'],
62         ];
63
64         $config['bootstrap'][] = 'gii';
65         $config['modules']['gii'] = [
66             'class' => 'yii\gii\Module',
67             // uncomment the following to add you
68             'allowedIPs' => ['*'],
69         ];
70     }
```

По умолчанию все пути у нас строятся через каталог «web», но мы можем это убрать, добавив параметр «**baseUrl**» с **пустым значением** в параметры «**request**», а также сразу добавим параметр для **русификации шаблонных сообщений фреймворка** в основную конфигурацию:

```
10     'aliases' => [
11         '@bower' => '@vendor/bower-asset',
12         '@npm'   => '@vendor/npm-asset',
13     ],
14     'language' => 'ru',
15     'components' => [
16         'request' => [
17             'baseUrl' => '',
18             'cookieValidationKey' => 'superSecretKey',
19         ],
```

Теперь мы можем проверить доступ к генератору, добавляем в наш путь «/gii» из **корневого домена (Внимание!!!** Если не сделать параметр «**baseUrl**», то в путь нужно добавить «/web/gii»). Должно выглядеть примерно так **http://fdgncpj-m1.wsr.ru/gii**

После чего у вас откроется интерфейс генератора. Нажмите на кнопку «Start» раздела «Model Generator» и вы увидите ошибку (это нормально, так как мы ещё не всё настроили):

Database Exception – yii\db\Exception

SQLSTATE[HY000] [1045] Access denied for user 'root'@'localhost' (using password: NO)

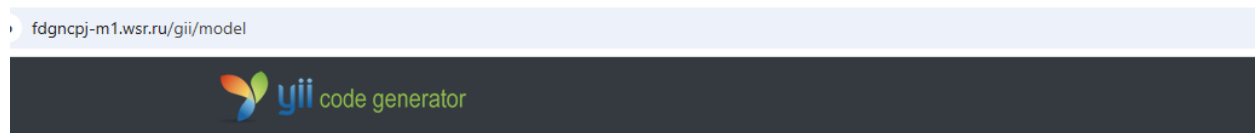
Error Info: Array

```
(
    [0] => HY000
    [1] => 1045
    [2] => Access denied for user 'root'@'localhost' (using password: NO)
)
```

Настроим подключение к базе данных (БД). В ЛК вы можете зайти в самую БД через «phpMyAdmin» (по умолчанию панель справа) и увидеть, что там уже создана архитектура и нам нужно будет только создавать таблицы. Откроем файл «**db.php**» в папке «**config**» и вставим туда только параметры: название нашей БД (**dbname**), логин (**username**) и пароль (**password**). **Остальные параметры не трогаем.**

```
rp  php db.php ×
<?php
return [
    'class' => 'yii\db\Connection',
    'dsn' => 'mysql:host=localhost;dbname=zxrysneh_m1',
    'username' => 'zxrysneh',
    'password' => 'a33fBm',
    'charset' => 'utf8',
```

Проверить правильность настройки можно, обновив страницу, где у нас открыт «Model Generator» и там появится интерфейс без ошибки:



На этом этапе можно сделать первый коммит, сохранив изменения. Вводим команду инициализации **пустого репозитория** через терминал, где **мы подключены по ssh**

```
zxrysneh@wsr:~/fdgncpj-m1$ git init
```

Теперь мы добавим все файлы в наш коммит, используя звёздочку (так как файл «.gitignore» исключит всё лишнее):

```
zxrysneh@wsr:~/fdgncpj-m1$ git add *
The following paths are ignored by one of your .gitignore files:
composer.lock
vendor
Use -f if you really want to add them.
```

Теперь мы сделаем первый коммит командой:

```
zxrysneh@wsr:~/fdgncpj-m1$ git commit -m "config project"
```

Внимание!!! Если появилось сообщение:

```
*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <zxrysneh@wsr.ru>) not allowed
```

То наш **коммит не прошёл** и нам нужно ещё сделать конфигурацию git, для этого копируем/вводим:

```
zxrysneh@wsr:~/fdgncpj-m1$ git config --global user.email "username@mail.com"
```

Затем копируем/вводим:

```
zxrysneh@wsr:~/fdgncpj-m1$ git config --global user.name "John Doe"
```

Делаем повторный коммит командой:

```
zxrysneh@wsr:~/fdgncpj-m1$ git commit -m "config project"
```

Проверить коммиты можно командой «reflog»:

```
zxrysneh@wsr:~/fdgncpj-m1$ git reflog
502828d (HEAD -> master) HEAD@{0}: commit (initial): config project
```

Внимание!!! Если по заданию будет требоваться название ветки «**main**», то перед тем как делать следующие коммиты нужно переименовать ветку командой:

```
zxrysneh@wsr:~/fdgncpj-m1$ git branch -m main
```

Можем проверить что ветка изменила название с «**master**» на «**main**» используя команду «**reflog**»:

```
zxrysneh@wsr:~/fdgncpj-m1$ git reflog
502828d (HEAD -> main) HEAD@{0}: Branch: renamed refs/heads/master to refs/heads/main
502828d (HEAD -> main) HEAD@{2}: commit (initial): config project
```

Дальше по проекту вы будете сами решать, когда делать коммиты. Нужно будет теперь выполнять последовательность команд:

- 1 `git add *`
- 2 `git commit -m "что изменили в проекте"`

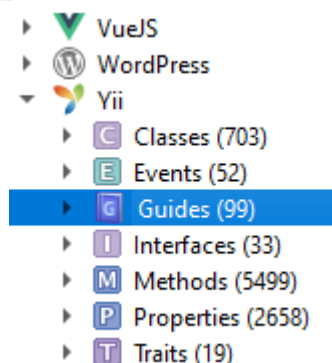
Создание БД

Начиная с этой главы и далее мы будем пользоваться онлайн справочником «Zeal», который поможет нам разрабатывать приложение, не запоминая наизусть код.

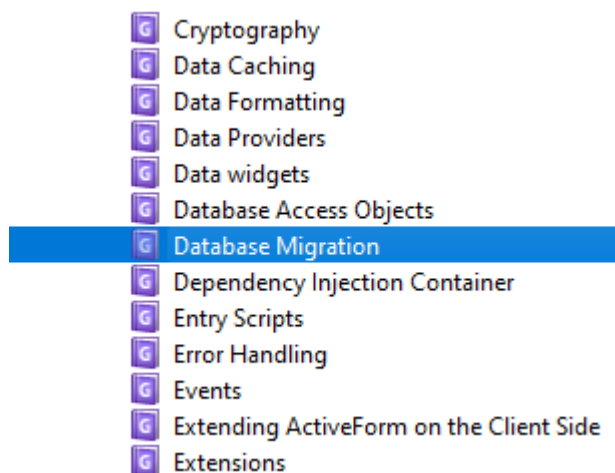
Так как фреймворк базируется на таблицах из баз данных, то следующим этапом мы должны сразу продумать нашу БД, на основе которой мы в будущем сгенерируем наш проект.

Рассмотрим создание таблиц через **миграции**. Если вы не хотите пользоваться данным способом, то листайте в конец данного раздела, где показана общая схема и можете создать всё тоже самое через интерфейс «phpMyAdmin».

Всё что мы будем использовать из «Zeal» расположено во вкладке «**Guides**» раздела по «**Yii**».



Сейчас нам нужно делать миграции, поэтому открываем справку «Database Migration»



Там мы видим, что типовая миграция будет выглядеть в виде строки: `yii migrate/create create_news_table`

Creating Migrations

To create a new migration, run the following command:

```
yii migrate/create <name>
```

The required `name` argument gives a brief description about the following command:

```
yii migrate/create create_news_table
```

Создадим первую миграцию для таблицы. Заходим в **терминал**, где **мы подключены по ssh** и вводим команду и подтверждаем миграцию (обратите внимание, что мы создали сначала таблицу для пользователей):

```
zxrysneh@wsr:~/fdgncpj-m1$ php yii migrate/create create_user_table
Yii Migration Tool (based on Yii v2.0.49)

Create new migration '/srv/users/zxrysneh/fdgncpj-m1/migrations/m250314_012526_create_user_table.php'? (yes|no) [no]:y
New migration created successfully.
```

Затем создадим миграцию для таблицы «service». **Внимание!!! Всегда сначала создаем таблицы с первичными ключами**, потом остальные, иначе вы получите ошибку при выполнении миграции.

```
zxrysneh@wsr:~/fdgncpj-m1$ php yii migrate/create create_service_table
```

Последнюю миграцию для таблицы «request» нам нужно создать с внешним ключом, поэтому мы сразу можем включить этот параметр в команду (если не сделаем это сразу, то можно добавить позже). В «Zeal» есть пример создания такого ключа.

Foreign keys

Since 2.0.8 the generator supports foreign keys using the `foreignKey` keyword.

```
yii migrate/create create_post_table --fields="author_id:integer:NotNull:foreignKey(user),category_id:integer:defaultValue(1):foreignKey,title:string,body:integer:defaultValue(1):foreignKey"
```

Мы её сократим, так как в примере создаётся больше зависимостей, чем нам нужно. Также необходимо учесть, что в заявке (по заданию) пользователь может указать не только конкретную услугу, а выбрать «иное» и ввести свой текст, поэтому разрешим оставлять это значение для «FK service_id» пустым:

```
zxrysneh@wsr:~/fdgncpj-m1$ php yii migrate/create create_request_table --fields="user_id:integer:NotNull:foreignKey(user),service_id:integer:foreignKey(service)"
```

Теперь нам надо указать какие поля и с какими типами данных будут содержаться в наших таблицах. Открываем в IDE PhpStorm папку «migrations», где созданы наши миграции:

```
▼ migrations
  © m250314_012526_create_user_table.php
  © m250314_012930_create_service_table.php
  © m250314_014232_create_request_table.php
```

Вы уже наверно обратили внимание, что в «Zeal» описаны примеры описания таблиц. Копируем/пишем по аналогии наши таблицы. По заданию у нас должен быть **уникальный логин**, остальные поля обязательны для заполнения, поэтому опишем нашу таблицу «user» так:

```
public function safeUp()
{
    $this->createTable( table: '{{%user}}', [
        'id' => $this->primaryKey(),
        'full_name' => $this->string( length: 100)->notNull(),
        'phone' => $this->string( length: 20)->notNull(),
        'email' => $this->string( length: 100)->notNull(),
        'username' => $this->string( length: 100)->notNull()->unique(),
        'password' => $this->string()->notNull(),
        'role' => "ENUM('user', 'admin') NOT NULL DEFAULT 'user'",
    ]);
}
```

Здесь сразу мы можем добавить в этот метод «seed» с данными **администратора**, взяв логин и пароль из задания.

```
    $this->insert( table: '{{%user}}', [
        'full_name' => 'admin',
        'phone' => '+7(999)999-99-99',
        'email' => 'admin@admin.com',
        'username' => 'adminka',
        'password' => md5( string: 'password'),
        'role' => 'admin',
    ]);
}
```

Таблица «service» будет у нас состоять всего из двух полей:

```
public function safeUp()
{
    $this->createTable( table: '{{%service}}', [
        'id' => $this->primaryKey(),
        'name' => $this->string( length: 50)->notNull(),
    ]);
}
```

Опять же сделаем заполнение этой таблицы через миграцию (обратите внимание на метод «**batchInsert**», который позволяет нам добавить несколько строк в колонку «**name**»):

```
$this->batchInsert( table: 'service', ['name'], [
    ['общий клининг'],
    ['генеральная уборка'],
    ['послестроительная уборка'],
    ['химчистка ковров и мебели']
]);

}
```

Осталось описать таблицу «request», которая будет выглядеть так:

```
public function safeUp()
{
    $this->createTable( table: '{{%request}}', [
        'id' => $this->primaryKey(),
        'user_id' => $this->integer()->notNull(),
        'service_id' => $this->integer(),
        'address' => $this->string()->notNull(),
        'contact_phone' => $this->string()->notNull(),
        'desired_datetime' => $this->dateTime()->notNull(),
        'custom_service_description' => $this->string(),
        'payment_method' => "ENUM('cash', 'card') NOT NULL",
        'status' => "ENUM('new', 'in_progress', 'completed', 'cancelled') DEFAULT 'new'",
        'cancellation_reason' => $this->string(),
    ]);
}
```

Внутри метода «**safeUp**» у вас должны быть описаны внешние зависимости, которые мы прописали в виде параметров при создании миграции. Если их нет, то можно посмотреть примеры в справке «Zeal» и описать их здесь напрямую:

```
$this->createIndex(
    name: '{{%idx-request-user_id}}',
    table: '{{%request}}',
    columns: 'user_id'
);

$this->addForeignKey(
    name: '{{%fk-request-user_id}}',
    table: '{{%request}}',
    columns: 'user_id',
    refTable: '{{%user}}',
    refColumns: 'id',
    delete: 'CASCADE'
);
```



```

$this->createIndex(
    name: '{{%idx-request-service_id}}',
    table: '{{%request}}',
    columns: 'service_id'
);

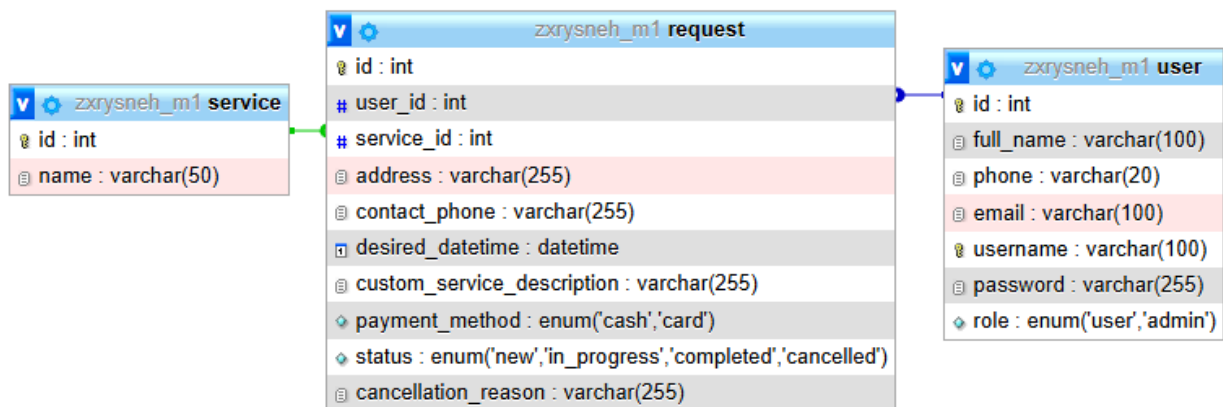
$this->addForeignKey(
    name: '{{%fk-request-service_id}}',
    table: '{{%request}}',
    columns: 'service_id',
    refTable: '{{%service}}',
    refColumns: 'id',
    delete: 'CASCADE'
);
}

```

Применим миграции и создадим таблицы в БД командой (написана в том же разделе в «Zeal»):

```
zxrystne@wsr:~/fdgncpj-m1$ php yii migrate
```

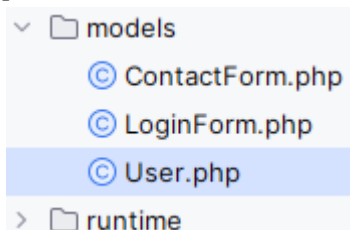
По итогу наша БД будет выглядеть так:



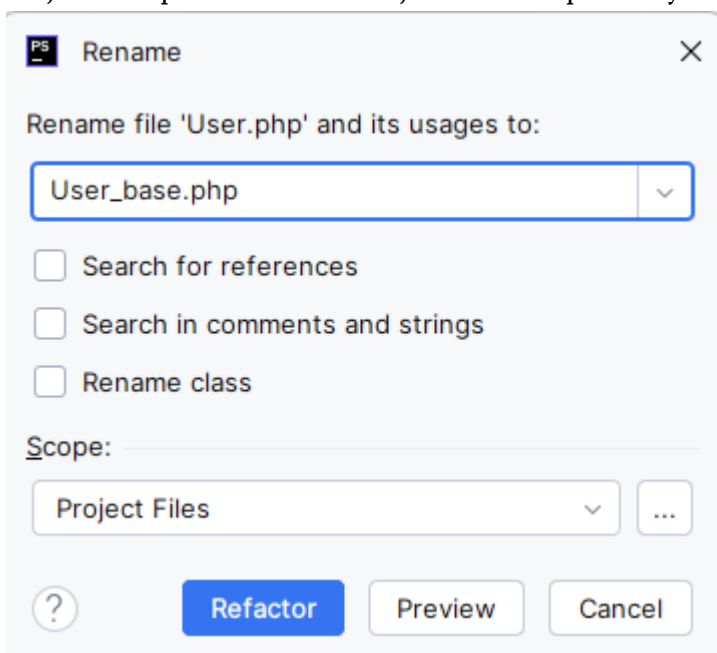
Внимание!!! Данная БД рассчитана просто под выполнение задания, автор осознаёт, что такие поля как «address» должны храниться в отдельной сущности, поэтому если вы считаете, что вам лучше подойдет другой набор таблиц, то хвалю вас и пожелаю удачи. Мы же будем работать с таким набором данных, так как его хватит для решения задания.

Авторизация пользователя

Перед тем как начать генерировать и писать код нам необходимо избавиться от стандартной модели пользователя:



Удалять мы её не будем, так как туда можно будет подглядывать, чтобы понимать набор методов для её реализации. Поэтому просто переименуем её **Shift+F6** или из контекстного меню «**Refactor - >Rename**». **Внимание! Обязательно убираем все галочки** перед тем, как сохранить новое имя, а то IDE переименует всё во всех файлах этой модели:



В браузере переходим в ГИ «Model Generator». Так как БД у нас уже сделана, то нажав на поле «Table Name» выбираем таблицу «user», имя модели должно подставиться само, если нет, то оно будет «User» и нажимаем «Preview», а затем «Generate»:

Table Name

user

☐ Standardize Capitals

☐ Singularize

Model Class Name

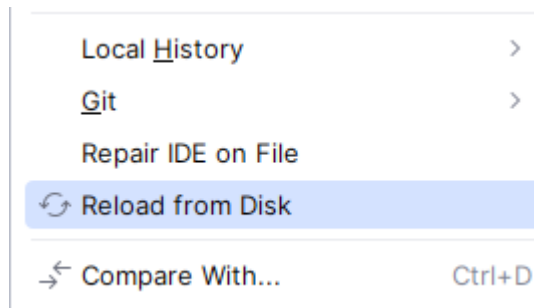
User

После генерации внизу появиться сообщение об успешной генерации:

The code has been generated successfully.

```
Generating code using template "/srv/users/zxryseh/fdgnnpj-m1/vendor/yiisoft/yii2-gii/src/generators/model/default/generated_models/User.php
done!
```

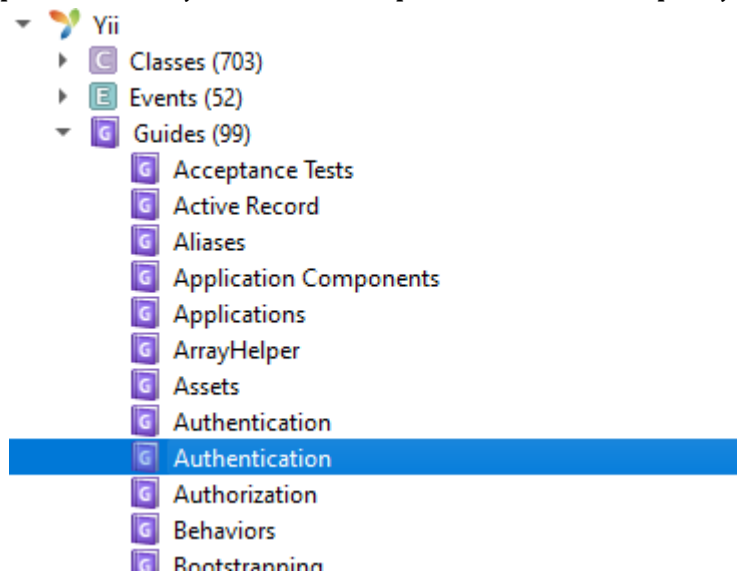
Переходим в PhpStorm и открываем нового «User» в папке «models». **Внимание!** Если файл не отображается, то нужно синхронизироваться с сервером. Вызываем контекст на папке «models» и нажимаем «Reload from Disk»:



Сразу **подключим интерфейс «IdentityInterface»** для нашей модели (IDE покажет, что у нас ошибка, подчеркнув строку и это нормально):

```
20 class User extends \yii\db\ActiveRecord implements \yii\web\IdentityInterface
21 {
```

Теперь нам нужно избавиться от ошибки, реализовав необходимые методы интерфейса «IdentityInterface». Открываем в «Zeal» справку «Authentication»:



Обратите внимание что там **две справки** с таким названием, но нам нужна в которой **больше текста**.

Нам нужен раздел имплементирования «IdentityInterface»:

Implementing [yii\web\IdentityInterface](#)

В примере мы видим, как должна быть реализована модель «User» через этот интерфейс, поэтому нам её нужно скопировать и вставить к нам (копируем только методы: **findIdentity**, **findIdentityByAccessToken**, **getId**, **getAuthKey** и **validateAuthKey**). Добавляем эти методы в конец нашего класса «User» (не трогаем остальные методы, они нам нужны):

```
70  public static function findIdentity($id)
71 {
72     return static::findOne($id);
73 }
74
75  public static function findIdentityByAccessToken($token, $type = null)
76 {
77     return static::findOne(['access_token' => $token]);
78 }
79
80  public function getId()
81 {
82     return $this->id;
83 }
84
85  public function getAuthKey()
86 {
87     return $this->auth_key;
88 }
89
90  public function validateAuthKey($authKey)
91 {
92     return $this->getAuthKey() === $authKey;
93 }
```

Стандартные комментарии удалены, вы их можете оставить. Ошибка на строке объявления класса «User» должна исчезнуть.

Теперь «заглушим» методы, которые нам не нужны в приложении. Для этого руководствуемся такой логикой: если в названии метода есть слова «Auth» и «Token», то они нам не нужны. Далее смотрим что они возвращают. Если данные (массив, объект, строка), то заменим на «null», если результат сравнения (булеан), то заменяем на «false». По итогу мы получим такой результат:

```

70  public static function findIdentity($id)
71  {
72      return static::findOne($id);
73  }
74
75  2 usages
76  public static function findIdentityByAccessToken($token, $type = null)
77  {
78      return null;
79  }
80
81  no usages
82  public function getId()
83  {
84      return $this->id;
85  }
86
87  no usages
88  public function getAuthKey()
89  {
90      return null;
91  }
92
93  2 usages
94  public function validateAuthKey($authKey)
95  {
96      return false;
97  }

```

Если мы посмотрим в базовую модель «User», то увидим там метод «findByUsername», который нам тоже нужен, но с более простой реализацией. Копируем его себе и удаляем его тело:

```

95  public static function findByUsername($username)
96  {
97
98  }

```

В справке листаем ниже до раздела «Using yii\web\User»:

- ASSETS
- Authentication
- Authentication
- Authorization
- Behaviors

Using yii\web\User

Нас интересует строка входа пользователя:

To login a user, you may use the following code:

```
// find a user identity with the specified username.  
// note that you may want to check the password if needed  
$identity = User::findOne(['username' => $username]);  
  
// logs in the user
```

Копируем её себе и переписываем её в более удобную нам запись:

```
95     public static function findByUsername($username)  
96     {  
97         return User::findOne(['username' => $username]);  
98     }
```

Осталось добавить сравнения наших паролей с учётом «md5», копируем из базовой модели метод «validatePassword» и добавляем его себе, но не забываем, что у нас пароль шифруется, поэтому перепишем код в такой вид:

```
100     public function validatePassword($password)  
101     {  
102         return $this->password === md5($password);  
103     }
```

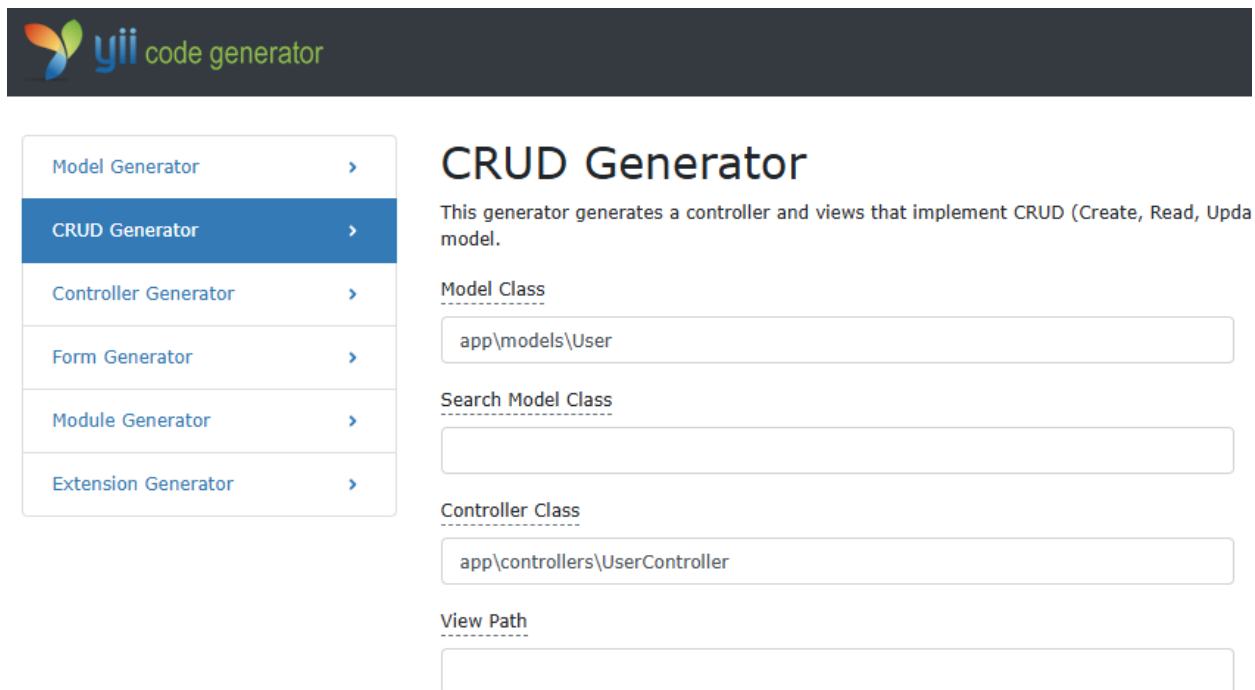
Теперь мы можем **авторизоваться** через вкладку «Login» нашего приложения, используя логин и пароль админа из задания (adminka и password).

Внимание!!! Не забудьте нажать «Logout» перед тем, как продолжить дальше работу, это может привести к ошибкам.

Если вы делали БД не через миграции, то вы можете просто вставить данные админа через интерфейс «phpMyAdmin», главное не забудьте поставить шифрование пароля в **md5**.

Регистрация и разработка интерфейса

Нужно сделать генерацию шаблонов и контролера через «CRUD Generator». Переходим в GII вкладку «CRUD Generator», нам необходимо заполнить всего два поля: «**Model Class**» и «**Controller Class**». Правильность заполнения можно увидеть при наведении на название поля.

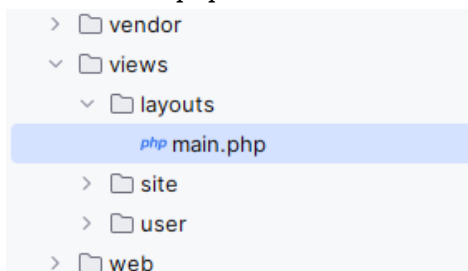


Можно сгенерировать всё, хоть нам и пригодиться только часть. Нажимаем «**Preview**» (если необходимо, то убираем галочки с представлений, которые мы не будем использовать), затем нажимаем «**Generate**» и появится консоль с сообщением успешной генерации:

The code has been generated successfully.

```
Generating code using template "/srv/users/zxryseh/fdgnnpj-m1/vendor/yiisoft/yii2-gii/src/generators/crud/default
generated controllers/UserController.php
generated views/user/_form.php
generated views/user/create.php
generated views/user/index.php
generated views/user/update.php
generated views/user/view.php
done!
```

Теперь уберем всё лишнее и создадим ссылку на регистрацию. Для начала можно изменить название нашего приложения. Открываем основное представление нашего приложения «main.php»:



Зажав Ctrl нажимаем мышкой по надписи «name», в котором храниться название нашего приложения:

```
32 <header id="header">
33     <?php
34     NavBar::begin([
35         'brandLabel' => Yii::$app->name,
36         'brandUrl' => Yii::$app->homeUrl,
37         'options' => ['class' => 'navbar-ex
38     ]);
```

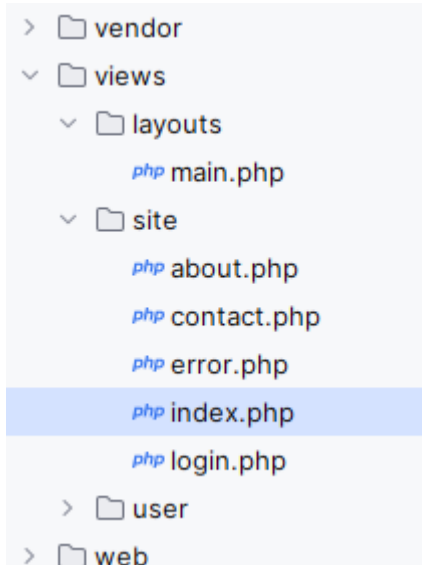
Изменяем значение из задания (PhpStorm выдаст предупреждение, но мы соглашаемся):

```
97     public $name = 'Мой Не Сам';
```

Либо мы можем задать это значение напрямую, сделав так (решайте сами как вам удобнее):

```
32 <header id="header">
33     <?php
34     NavBar::begin([
35         'brandLabel' => 'Мой Не Сам',
36         'brandUrl' => Yii::$app->homeUrl,
37         'options' => ['class' => 'navbar-
```

Вы можете изменить «title» для главной страницы (для всех страниц аналогично). Переходим в index.php:



В самом верху изменяем параметр:

```
5     $this->title = 'Мой Не Сам';
```


Возвращаемся в файл «main.php» и удалим лишние строки, добавив ссылку на представление нашей регистрации (обратите внимание что через параметр «label» мы русифицируем текст ссылки):

```

39     echo Nav::widget([
40         'options' => ['class' => 'navbar-nav'],
41         'items' => [
42             ['label' => 'Регистрация', 'url' => ['/user/create']],
43             Yii::$app->user->isGuest
44             ? ['label' => 'Login', 'url' => ['/site/login']]

```

Так как она **должны** **будет** **скрываться**, когда пользователь авторизуется, то добавим сразу флаг, привязав его через параметр видимости в зависимости, описанную в этом же файле на следующей строке:

```

echo Nav::widget([
    'options' => ['class' => 'navbar-nav'],
    'items' => [
        ['label' => 'Регистрация', 'url' => ['/user/create'], 'visible' => Yii::$app->user->isGuest],
        Yii::$app->user->isGuest

```

Сразу русифицируем кнопки «Вход» и «Выход»:

```

'items' => [
    ['label' => 'Регистрация', 'url' =>
        Yii::$app->user->isGuest
        ? ['label' => 'Вход', 'url' =>
            : '<li class="nav-item">
                . Html::beginForm(['/site/login'])
                . Html::submitButton(
                    content: 'Выход (' . Yii::$app->user->isGuest ? 'Вход' : 'Выход') . ')',
                    ['class' => 'nav-link btn btn-light']

```

В этом же файле можно настроить подвал нашего сайта, например вот так:

```

69 <footer id="footer" class="mt-auto py-3 bg-light">
70     <div class="container">
71         <div class="row text-muted">
72             <div class="col-md-6 text-center text-md-start">&laquo;Мой Не Сам&raquo; <?= date( format: 'Y') ?></div>
73         </div>
74     </div>

```

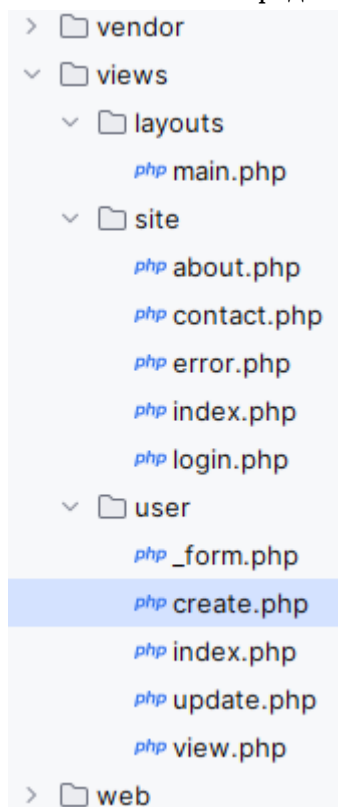
Если вам нужно убрать с сайта «хлебные крошки», то просто закомментируйте/удалите:

```

59 <main id="main" class="flex-shrink-0" role="main">
60     <div class="container">
61         <!-- --><?php //if (!empty($this->params['breadcrumbs'])): ?>
62         <!-- --><?php //= Breadcrumbs::widget(['links' => $this->params['breadcrumbs']]) ?>
63         <!-- --><?php //endif ?>
64         <?= Alert::widget() ?>
65         <?= $content ?>
66     </div>
67 </main>

```

Приступим к интерфейсу «Авторизации», для начала изменим надпись через свойство «title» нашего представления:

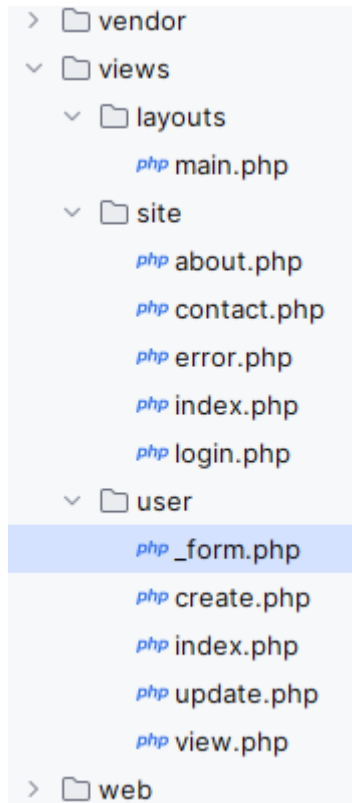


```
7
8 $this->title = 'Регистрация';
9 $this->params['breadcrumbs'][]
10 $this->params['breadcrumbs'][]
```

Для русификации подписи полей регистрации изменим их в модели пользователя (models -> user.php), удалив поле роли, так как оно задано по умолчанию:

```
48 public function attributeLabels()
49 {
50     return [
51         'id' => 'ID',
52         'full_name' => 'ФИО',
53         'phone' => 'Телефон',
54         'email' => 'Email',
55         'username' => 'Логин',
56         'password' => 'Пароль',
57     ];
58 }
```

Удалим ещё отображение поле роли в форме, переходим в файл «_form.php»:



Находим там строчку с ролью и удаляем её и меняем надпись на кнопке:

```
11 <div class="user-form">
12
13     <?php $form = ActiveForm::begin(); ?>
14
15     <?= $form->field($model, attribute: 'full_name')->textInput(['maxlength' => true]) ?>
16
17     <?= $form->field($model, attribute: 'phone')->textInput(['maxlength' => true]) ?>
18
19     <?= $form->field($model, attribute: 'email')->textInput(['maxlength' => true]) ?>
20
21     <?= $form->field($model, attribute: 'username')->textInput(['maxlength' => true]) ?>
22
23     <?= $form->field($model, attribute: 'password')->passwordInput(['maxlength' => true]) ?>
24
25     <div class="form-group">
26         <?= Html::submitButton( content: 'Регистрация', ['class' => 'btn btn-success']) ?>
27     </div>
```

Стоит упомянуть, что русифицировать подписи полей можно и через этот файл, вызвав дополнительное свойство «label» и записать туда новое значение, например:

```
<?= $form->field($model, attribute: 'full_name')->textInput(['maxlength' => true])->label( label: 'ФИО') ?>
```

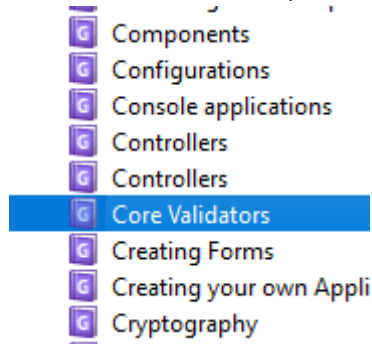
Изменим правила валидации, которые требуются по заданию. Стандартно они находятся в модели пользователя и выглядят сейчас так:

```

33 public function rules()
34 {
35     return [
36         [['full_name', 'phone', 'email', 'username', 'password'], 'required'],
37         [['role'], 'string'],
38         [['full_name', 'email', 'username'], 'string', 'max' => 100],
39         [['phone'], 'string', 'max' => 20],
40         [['password'], 'string', 'max' => 255],
41         [['username'], 'unique'],
42     ];
43 }

```

Почти всё что нам нужно опять берём из «Zeal» в теме «Core Validators»



Хоть у нас и стоит в БД, что роль задана по умолчанию, но посмотрим как прописать это в модели. Смотрим в раздел «default»:

default

```

[
    // set "age" to be null if it is empty
    ['age', 'default', 'value' => null],

    // set "country" to be "USA" if it is empty
    ['country', 'default', 'value' => 'USA'],

    // assign "from" and "to" with a date 3 days and 6 days from today, if they are empty
    [['from', 'to'], 'default', 'value' => function ($model, $attribute) {
        return date('Y-m-d', strtotime($attribute === 'to' ? '+3 days' : '+6 days'));
    }],
]

```

По аналогии прописываем себе новое правило для роли:

```
['role', 'default', 'value' => 'user'],
```

Далее добавим из справки себе правило валидации адреса электронной почты:

email

```
[
    // checks if "email" is
    ['email', 'email'],
]
```

Листая справку, ниже находим ещё одну валидацию, которую мы изменим:

match

```
[
    // checks if "username" starts with a letter and con
    ['username', 'match', 'pattern' => '/^[a-z]\w*$/i']
]
```

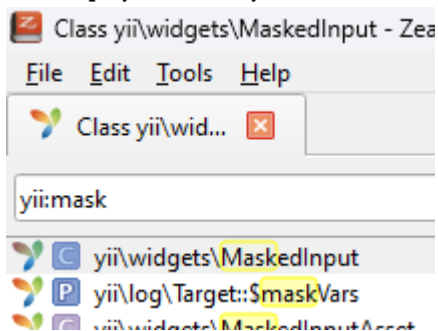
Можете добавить это правило себе на валидацию логина (**username**), но оно не требуется по заданию, добавим себе только верхний регистр и получим такую строку:

```
['username', 'match', 'pattern' => '/^[A-Z]\w*$/i']
```

Для валидации поля «ФИО» нам нужно скопировать и изменить правило (добавлено тире, хотя по заданию не требуется):

```
['full_name', 'match', 'pattern' => '/^[A-яЁё -]*$/u'],
```

Валидация телефона имеет довольно сложный паттерн, но мы можем его обойти с помощью маски ввода. Ищем в справке эту маску. Достаточно в поиск ввести такую строку и открыть первую ссылку:



Нам нужно обратить внимание на виджет маски и просто скопировать его себе, изменив под наш формат:

You can also use this widget in an [yii\widgets\ActiveForm](#) using the [widget\(\)](#) method, for example like this:

```
<?= $form->field($model, 'from_date')->widget(\yii\widgets\MaskedInput::class, [
    'mask' => '999-999-9999',
]) ?>
```

Копируем себе это в файл «**_form.php**», заменив наше поле «phone». Меняем маску под себя и получаем такой результат (не забудьте изменить привязку параметра «form_date» на «phone»):

```
13     <?php $form = ActiveForm::begin(); ?>
14
15     <?= $form->field($model, attribute: 'full_name')->textInput(['maxlength' => true]) ?>
16
17     <?= $form->field($model, attribute: 'phone')->widget( class: \yii\widgets\MaskedInput::class, [
18         'mask' => '+7(999)-999-99-99',
19     ]) ?>
20
21     <?= $form->field($model, attribute: 'email')->textInput(['maxlength' => true]) ?>
22
```

Вообще для того, чтобы убедиться, что пользователь точно ввёл все цифры нужно добавить правило валидации нашего номера, которое будет выглядеть так:

```
['phone', 'match', 'pattern' => '/^\\+?7\\(\\d{3}\\)-\\d{3}-\\d{2}-\\d{2}$/',
```

Не забываем, что у пароля по заданию минимум 6 символов, поэтому дополним основное правило валидации:

```
[['password'], 'string', 'max' => 255, 'min' => 6],
```

Теперь изменим правило переадресации при успешной регистрации на страницу авторизации. Для этого открываем файл «**UserController.php**» в папке «**Controllers**». Находим метод «actionCreate» и меняем правило редиректа (redirect) на наше представление авторизации:

```
78     public function actionCreate()
79     {
80         $model = new User();
81
82         if ($this->request->isPost) {
83             if ($model->load($this->request->post()) && $model->save()) {
84                 return $this->redirect(['site/login']);
85             }
86         } else {
87             $model->loadDefaultValues();
88         }
89     }
90
```

У нас осталось добавить метод для шифрования нашего пароля перед сохранением. Добавим метод «beforeSave» в нашу модель «User». Перехватив данные зашифруем, и передадим на обработку родительского класса. Выглядеть это будет так:

```
110  @↑ public function beforeSave($insert)
111      {
112          $this->password = md5($this->password);
113          return parent::beforeSave($insert);
114      }
115
```

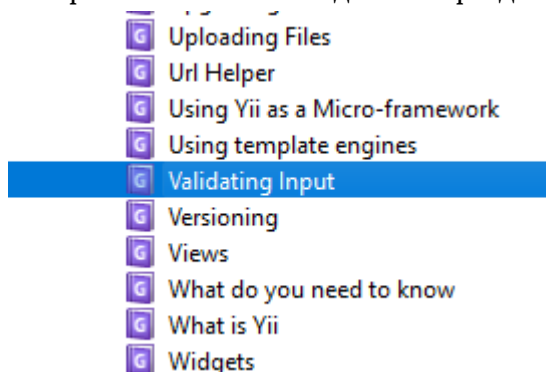
После этого можете проверить что у вас **работает полностью функционал регистрации**. Правила валидации у вас по итогу могут выглядеть так (меняем под задание по необходимости):

```
33 public function rules()
34 {
35     return [
36         [['full_name', 'phone', 'email', 'username', 'password'], 'required'],
37         [['role'], 'string'],
38         ['role', 'default', 'value' => 'user'],
39         [['full_name', 'email', 'username'], 'string', 'max' => 100],
40         [['phone'], 'string', 'max' => 20],
41         [['password'], 'string', 'max' => 255, 'min' => 6],
42         [['username'], 'unique'],
43         ['email', 'email'],
44         ['username', 'match', 'pattern' => '/^[A-z]\w*$/i'],
45         ['full_name', 'match', 'pattern' => '/^[A-яЁё -]*$/u'],
46         ['phone', 'match', 'pattern' => '/^\+?7(\d{3})-\d{3}-\d{2}-\d{2}$/'],
47     ];
48 }
```

Если вам не нравятся **встроенные сообщения об ошибке** или это требуется по заданию, то их можно тоже изменить через параметр «message», например зададим своё сообщение для ошибки валидации поля «ФИО»:

```
['full_name', 'match', 'pattern' => '/^[A-яЁё -]*$/u',
    'message' => "Разрешены только кириллица, пробел и тире"],
```

В справочнике это находится в разделе «**Validating Input**»



Дополнительно. По заданию это не требуется, но лучше проверять уникальность логина сразу после ввода. Для этого можно использовать встроенный **AJAX**. Необходимая нам информации находится в том же разделе «**Validating Input**» (смотрите скрин выше).

Находим тему «**AJAX Validation**»

AJAX Validation

Some validations can only be done on the server-side, because only the server has the necessary information. You can use AJAX-based validation in this case. It will trigger an AJAX request in the background.

To enable AJAX validation for a single input field, configure the `enableAjaxValidation` property of that field.

```
use yii\widgets\ActiveForm;

$form = ActiveForm::begin([
    'id' => 'registration-form',
]);

echo $form->field($model, 'username', ['enableAjaxValidation' => true]);

// ...

ActiveForm::end();
```

Из примера смотрим, что нужно включить AJAX на нашей форме (можно на отдельном поле, если хотите).

To enable AJAX validation for all inputs of the form, configure `enableAjaxValidation` to be `true` at the form level:

```
$form = ActiveForm::begin([
    'id' => 'contact-form',
    'enableAjaxValidation' => true,
]);
```

Открываем наш файл «`_form.php`» и добавляем это правило

```
11 <div class="user-form">
12
13 <?php $form = ActiveForm::begin(['enableAjaxValidation' => true]); ?>
14
15 <?= $form->field($model, attribute: 'full_name')->textInput(['maxlength' => true]) ?>
```

Ниже есть код, который мы должны добавить в наш контроллер

You also need to prepare the server so that it can handle the AJAX validation requests. This can be achieved by a

```
if (Yii::$app->request->isAjax && $model->load(Yii::$app->request->post())) {
    Yii::$app->response->format = Response::FORMAT_JSON;
    return ActiveForm::validate($model);
}
```

Добавляем это правило в наш «`UserController.php`» в метод «`actionCreate`»

```
78 public function actionCreate()
79 {
80     $model = new User();
81     if (Yii::$app->request->isAjax && $model->load(Yii::$app->request->post())) {
82         Yii::$app->response->format = Response::FORMAT_JSON;
83         return ActiveForm::validate($model);
84     }
85     if ($this->request->isPost) {
86         if ($model->load($this->request->post()) && $model->save()) {
87             return $this->redirect(['site/login']);
88         }
89     }
90 }
```

PhpStorm выделит вам поля, которые нужно импортировать. Нажимаем на надпись «`Yii`» (их там много, и они выделены фоном)


```

80         $model = new User();
81         if (Yii::$app->request->isAjax) {
82             Yii::$app->response->format = Response::FORMAT_JSON;

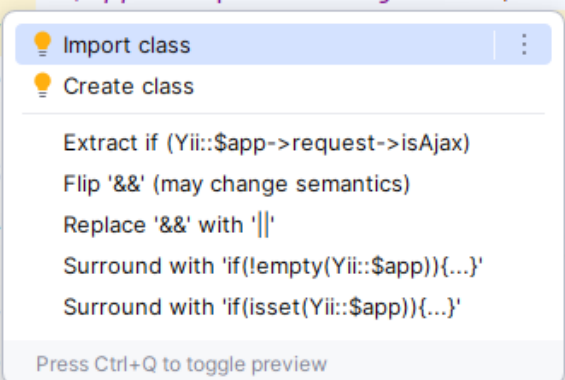
```

Вызываем контекстное меню IDE сочетанием **Alt+Enter**

```

80         $model = new User();
81         if (Yii::$app->request->isAjax) {
82             Yii::$app->response->format = Response::FORMAT_JSON;
83             return ActiveForm::validate($model);
84         }
85         if ($this->request->isPost) {
86             if ($model->load($this->request->post()) && $model->loadDefaultValues()) {
87                 return $this->redirect(['site/index']);
88             }
89         } else {

```



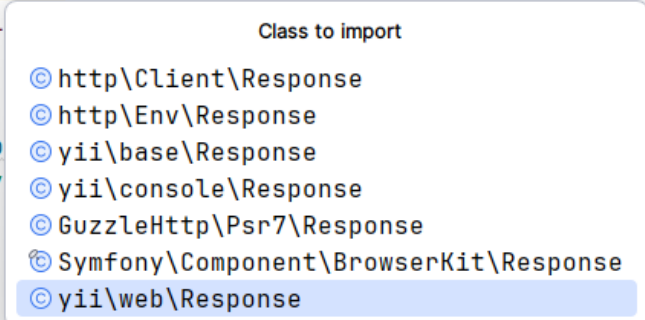
Нам нужен как раз первый пункт «**Import class**». То есть просто нажимаем «**Enter**»

Аналогично нужно сделать для класса «**Response**», но там появиться выбор из множества, в нашем случае выбираем «**yii\web\Response**»

```

if (Yii::$app->request->isAjax) {
    Yii::$app->response->format = Response::FORMAT_JSON;
    return ActiveForm::validate($model);
}
if ($this->request->isPost) {
    if ($model->load($this->request->post()) && $model->loadDefaultValues()) {
        return $this->redirect(['site/index']);
    }
} else {
    $model->loadDefaultValues();
}

```

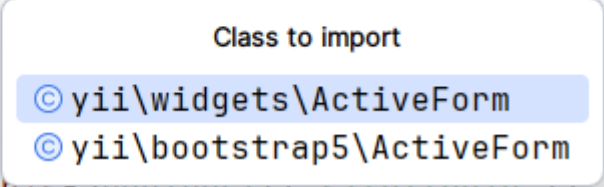


Для класса «**ActiveForm**» нам подходит первый вариант «**yii\widgets\ActiveForm**»

```

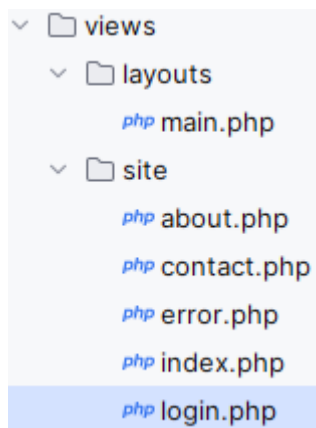
if (Yii::$app->request->isAjax) {
    Yii::$app->response->format = Response::FORMAT_JSON;
    return ActiveForm::validate($model);
}
if ($this->request->isPost) {
    if ($model->load($this->request->post()) && $model->loadDefaultValues()) {
        return $this->redirect(['site/index']);
    }
} else {
    $model->loadDefaultValues();
}

```



Скорее всего **PhpStorm** начнёт выделять всю строку, в которой мы возвращаем результат, но можете это **игнорировать**. Всё будет работать. Уникальность логина теперь будет проверяться сразу после ввода.

Внимание! Мы не рассмотрели русификацию страницы авторизации, но по аналогии вы уже можете её сами перевести. Все находится в файле «login.php»



Вот пример полностью переведенной страницы авторизации (также убрано сообщение о «пользователях заглушках»):

```
11 $this->title = 'Авторизация';
12 $this->params['breadcrumbs'][] = $this->title;
13 ?>
14 <div class="site-login">
15     <h1><?= Html::encode($this->title) ?></h1>
16     <p>Пожалуйста введите логин и пароль для авторизации:</p>
17     <div class="row">
18         <div class="col-lg-5">
19             <?php $form = ActiveForm::begin([
20                 'id' => 'login-form',
21                 'fieldConfig' => [
22                     'template' => "{label}\n{input}\n{error}",
23                     'labelOptions' => ['class' => 'col-lg-1 col-form-label mr-lg-3'],
24                     'inputOptions' => ['class' => 'col-lg-3 form-control'],
25                     'errorOptions' => ['class' => 'col-lg-7 invalid-feedback'],
26                 ],
27             ]); ?>
28             <?= $form->field($model, attribute: 'username')->textInput(['autofocus' => true])->label('Логин') ?>
29             <?= $form->field($model, attribute: 'password')->passwordInput()->label('Пароль') ?>
30             <?= $form->field($model, attribute: 'rememberMe')->checkbox([
31                 'template' => "<div class=\"custom-control custom-checkbox\">{input} {label}</div>\n<div class=\"col-lg-8\">
32             ])->label('Запомнить меня') ?>
33             <div class="form-group">
34                 <div>
35                     <?= Html::submitButton( content: 'Вход', ['class' => 'btn btn-primary', 'name' => 'login-button']) ?>
36                 </div>
37             </div>
38             <?php ActiveForm::end(); ?>
39         </div>
40     </div>
41 </div>
42 </div>
```

Создание и обработка заявок

Создание правил для заявок

Для работы необходимо сгенерировать оставшиеся **модели** для наших сущностей «**service**» и «**request**», которые помогут нам реализовать оставшуюся логику сайта.

Переходим в раздел «**Model Generator**» нашего код генератора «**yii**» и создаём модели для сущностей, аналогично как мы делали для «**user**»

Model Generator

This generator generates an ActiveRecord class for the specified database table.

Database Connection ID

db

☐ Use Table Prefix

☒ Use Schema Name

Table Name

service

☐ Standardize Capitals

☐ Singularize

Model Class Name

Service

В результате для двух оставшихся таблиц мы должны получить **сгенерированные модели**:

The code has been generated successfully.

```
Generating code using template "/srv/users/zxryseh/fdgnpcj-m1/vendor/yiisoft/yii2-gii/src/generators/model/default/generated_models/Request.php
done!
```

Для работы нам также потребуется генерация из вкладки «**CRUD**» для «**request**». Указываем только ссылку на **модель** и **контроллер**, так как **модель** поиска нам не нужна, а **представления** будут сгенерированы автоматически.

CRUD Generator

This generator generates a controller and views that implement CRUD (Create, Read, Update model).

Model Class

app\models\Request

Search Model Class

Controller Class

app\controllers\RequestController

View Path

Добавим **отображение наших заявок для авторизованного пользователя** в файл «**main.php**» в папке «**views/layouts**». Обратите внимание на «**!**» перед параметром отображения нашего меню.

```
echo Nav::widget([
    'options' => ['class' => 'navbar-nav'],
    'items' => [
        ['label' => 'Регистрация', 'url' => ['/user/create'], 'visible' => Yii::$app->user->isGuest],
        ['label' => 'Мои заявки', 'url' => ['/request/index'], 'visible' => !Yii::$app->user->isGuest],
        Yii::$app->user->isGuest
        ? ['label' => 'Вход', 'url' => ['/site/login']]
        : '<li class="nav-item">'
            . Html::beginForm(['/site/logout'])
            . Html::submitButton(
```

Правильнее организовать логику через метод «**behaviors**», который проверят авторизацию и роль пользователя, но мы сделаем проще. Проверим роль пользователя через свой метод «**isAdmin**», который напишем в нашей модели «**User.php**»

```
public function isAdmin()
{
    return $this->role == 'admin';
}
```

Теперь нужно **ограничить доступы только для авторизованных пользователей**. Каждый пользователь должен видеть только свои заявки, а администратор все, но при этом он не должен иметь возможности создавать самостоятельно заявки.

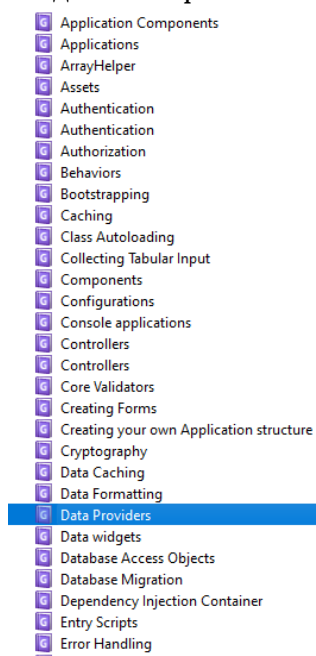
Визуально организуем так, чтобы пункты меню для пользователя и администратора выглядел по-разному, например можно сделать так в «views/layouts/main.php»:

```
41     'items' => [  
42         ['label' => 'Регистрация', 'url' => ['/user/create'], 'visible' => Yii::$app->user->isGuest],  
43         ['label' => 'Мои заявки', 'url' => ['/request/index'], 'visible' =>  
44             !Yii::$app->user->isGuest && !Yii::$app->user->identity->isAdmin()],  
45         ['label' => 'Заявки', 'url' => ['/request/index'], 'visible' =>  
46             !Yii::$app->user->isGuest && Yii::$app->user->identity->isAdmin()],  
47         Yii::$app->user->isGuest  
48         ? ['label' => 'Вход', 'url' => ['/site/login']]
```

Можно после авторизации также пользователя перенаправлять на заявки, написав в основной контроллер «SiteController.php» переадресацию на заявки:

```
72     public function actionLogin()  
73     {  
74         if (!Yii::$app->user->isGuest) {  
75             return $this->goHome();  
76         }  
77  
78         $model = new LoginForm();  
79         if ($model->load(Yii::$app->request->post()) && $model->login()) {  
80             return $this->redirect('url' => ['/request/index']);  
81         }  
82  
83         $model->password = '';  
84         return $this->render('view' => 'login', [  
85             'model' => $model,  
86         ]);  
87     }
```

У нас теперь пользователь видит все заявки, но их нужно разграничивать, для этого нам пригодится справка в «Zeal»



Active Data Provider

To use [yii\data\ActiveDataProvider](#), you should configure its [query](#) property, either arrays or [Active Record](#) instances. For example,

```
use yii\data\ActiveDataProvider;  
  
$query = Post::find()->where(['status' => 1]);  
  
$provider = new ActiveDataProvider([  
    'query' => $query,  
    'pagination' => [  
        'pageSize' => 10,  
    ],  
    'sort' => [  
        'defaultOrder' => [  
            'created_at' => SORT_DESC,  
            'title' => SORT_ASC,  
        ],  
    ],  
]);  
  
// returns an array of Post objects  
$posts = $provider->getModels();
```

Открываем файл «RequestController.php». Нас там интересует метод «actionIndex()», в котором возвращается метод «render», но там находит **всё заявки** (подойдёт для администратора). А вот для пользователя мы добавим в запрос ограничение по «user_id», взяв его из **текущей сессии пользователя**. Перепишем метод в такой вид, исходя из первоначального метода и справки «Zeal»:

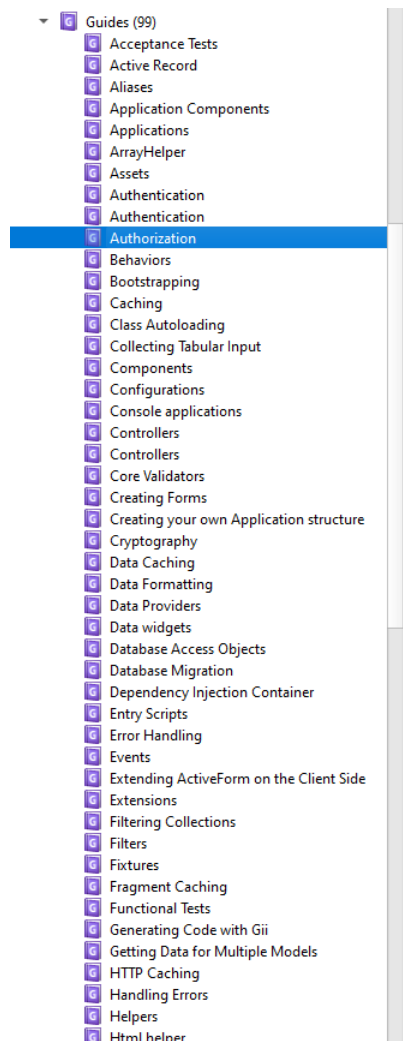
```
40     public function actionIndex()
41     {
42         if (Yii::$app->user->identity->isAdmin()) {
43             $dataProvider = new ActiveDataProvider([
44                 'query' => Request::find(),
45             ]);
46         } else {
47             $dataProvider = new ActiveDataProvider([
48                 'query' => Request::find()
49                 ->where(['user_id' => Yii::$app->user->id])
50             ]);
51         }
52         return $this->render( view: 'index', [
53             'dataProvider' => $dataProvider,
54         ]);
55     }
```

В этом же методе мы можем **настроить переадресацию неавторизованного пользователя на форму авторизации**, но опять же это не очень хороший способ (ниже рассмотрим более правильный способ). Вот так мы можем сделать, для того чтобы это начало работать:

```
40     public function actionIndex()
41     {
42         if (Yii::$app->user->isGuest) {
43             return $this->redirect(['site/login']);
44         }
45         else if (Yii::$app->user->identity->isAdmin()) {
46             $dataProvider = new ActiveDataProvider([
47                 'query' => Request::find(),
```

Внимание!!! Данный способ является более верным и красивым, но не обязателен, так как в задании нет прямых указаний на это.

Для того чтобы всё настроить через метод «behaviors» находим в справке «Zeal»:



Authorization

Authorization is the process of verifying that a user has enough permission to do something. Yii provides

Access Control Filter

Access Control Filter (ACF) is a simple authorization method implemented as [yii\filters\AccessControlFilter](#) that can be used in a controller or a module. While a user is requesting to execute an action, ACF

The code below shows how to use ACF in the `site` controller:

```
use yii\web\Controller;
use yii\filters\AccessControl;

class SiteController extends Controller
{
    public function behaviors()
    {
        return [
            'access' => [
                'class' => AccessControl::class,
                'only' => ['login', 'logout', 'signup'],
                'rules' => [
                    [
                        'allow' => true,
                        'actions' => ['login', 'signup'],
                        'roles' => ['?'],
                    ],
                    [
                        'allow' => true,
                        'actions' => ['logout'],
                        'roles' => ['@'],
                    ],
                ],
            ],
        ];
    }
    // ...
}
```

Но в примере из справки **слишком много правил** и нам достаточно реализовать только **доступ авторизированных пользователей**, а стандартный обработчик будет **автоматически производить переадресацию** на страницу авторизации. Перепишем метод «behaviors» в нашем контроллере «RequestController.php»

```
20 public function behaviors()
21 {
22     return [
23         'access' => [
24             'class' => \yii\filters\AccessControl::class,
25             'rules' => [
26                 [
27                     'allow' => true,
28                     'roles' => ['@'], // Только авторизованные пользователи
29                 ],
30             ],
31         ],
32     ];
33 }
```

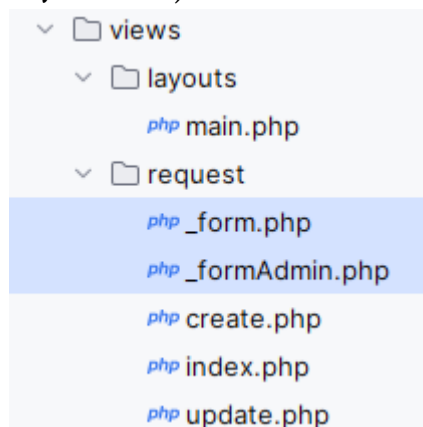
Ограничим добавление заявок для администратора, просто скрыв кнопку подачи заявки. Заменяем ещё заголовок через тернарный оператор в зависимости от роли. Записываем всё в «views/request/index.php»

```
11
12 $this->title = Yii::$app->user->identity->isAdmin() ? 'Заявки' : 'Мои заявки';
13 $this->params['breadcrumbs'][] = $this->title;
14 ?>
15 <div class="request-index">
16
17     <h1><?= Html::encode($this->title) ?></h1>
18
19     <?php if (!Yii::$app->user->identity->isAdmin()) { ?>
20         <p>
21             <?= Html::a( text: 'Подать заявку', ['create'], ['class' => 'btn btn-success']) ?>
22         </p>
23     <?php } ?>
24
```

Переведём надпись для формы заявки через «views/request/create.php»

```
7
8 $this->title = 'Подать заявку';
9 $this->params['breadcrumbs'][] = ['label' => 'Requests', 'url' => ['index']];
10 $this->params['breadcrumbs'][] = $this->title;
11 ?>
12 <div class="request-create">
```

Нам нужно оставить только нужные поля ввода для заявки, поэтому нам нужно изменить стандартное представление формы заявки «views/request/_form.php», также чтобы не создавать форму потом с нуля для администратора, сдублируем её и назовём «_formAdmin.php» (можно всё отображение привязать к роли и сделать условную отрисовку от этого)



Создание функционала заявок пользователя

Будем поля отображать в порядке, как в задании и делать отрисовку. Первое поле «**user_id**» мы вообще **удалим**, так как его будем брать из текущей сессии пользователя. По заданию первым полем должен быть адрес, поэтому просто перенесём его выше (правило обязательности должно быть из генерации, если его нет, то пропишите в модели «**Request.php**»)

```
13      <?php $form = ActiveForm::begin(); ?>
14
15      <?= $form->field($model, attribute: 'address')->textInput(['maxlength' => true]) ?>
16
17      <?= $form->field($model, attribute: 'service_id')->textInput() ?>
```

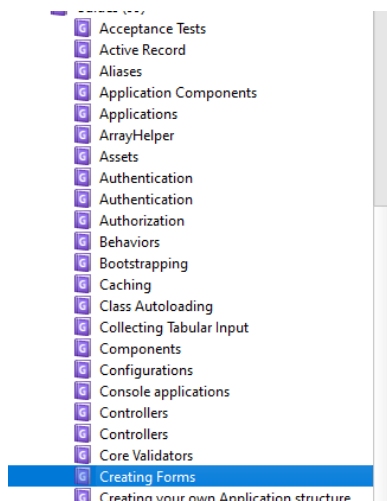
Следующее поле должно быть **номером телефона**, которое мы должны дать ввести пользователю, но мы уже делали **виджет с маской** для поля ввода номера телефона при регистрации (а ещё мы уже знаем номер нашего пользователя), поэтому копируем код виджета номера из «**views/user/_form.php**» к нам, **заменяем атрибут** на «**contact_phone**» (**не забудьте удалить стандартное поле ввода телефона**), а ещё подставим номер, который пользователь указал при регистрации (**можете добавить валидацию для номера из нашей модели пользователя в нашу модель заявки**)

```
15      <?= $form->field($model, attribute: 'address')->textInput(['maxlength' => true]) ?>
16
17      <?= $form->field($model, attribute: 'contact_phone')->widget( class: \yii\widgets\MaskedInput::class, [
18          'mask' => '+7(999)-999-99-99', 'options' => ['value' => Yii::$app->user->identity->phone]
19      ]) ?>
20
21      <?= $form->field($model, attribute: 'service_id')->textInput() ?>
```

Желаемую дату и время мы используем из стандартного набора полей ввода в таком виде

```
17      <?= $form->field($model, attribute: 'contact_phone')->widget( class: \yii\widgets\MaskedInput::class, [
18          'mask' => '+7(999)-999-99-99', 'options' => ['value' => Yii::$app->user->identity->phone]
19      ]) ?>
20
21      <?= $form->field($model, attribute: 'desired_datetime')->input( type: 'datetime-local') ?>
22
23      <?= $form->field($model, attribute: 'service_id')->textInput() ?>
```

Для выбора услуги воспользуемся в справке «**Zeal**» статьёй «**Creating Forms**», нас интересует **выпадающий список** («**Creating Lists**»)



Creating Lists

There are 3 types of lists:

- Dropdown lists
- Radio lists
- Checkbox lists

To create a list, you have to prepare the items. This can be done manually:

```
$items = [
    1 => 'item 1',
    2 => 'item 2'
]
```

or by retrieval from the DB:

Соберём все возможные услуги в один массив в переменной «**\$services**», добавив код в начале формы (вообще можно это сделать перед отображением списка).

```
13 <?php $form = ActiveForm::begin();
14 $services = \app\models\Service::find()
15     ->select(['name'])
16     ->indexBy( column: 'id')
17     ->column();
18 ?>
```

И отобразим этот список пользователю, передав нашу переменную в качестве аргумента списка, «Yii2» всё выведет как надо, так как данные взяты из БД.

```
26 <?= $form->field($model, attribute: 'desired_datetime')->input( type: 'datetime-local') ?>
27
28 <?= $form->field($model, attribute: 'service_id')->dropDownList($services) ?>
29
30 <?= $form->field($model, attribute: 'custom_service_description')->textInput(['maxlength' => true]) ?>
```

Сначала добавим чекбокс для выбора иной услуги в нашу форму

```
28 <?= $form->field($model, attribute: 'service_id')->dropDownList($services) ?>
29
30 <?= $form->field($model, attribute: 'other_request')->checkbox() ?>
31
32 <?= $form->field($model, attribute: 'custom_service_description')->textInput(['maxlength' => true]) ?>
```

Так как это не поле из нашей БД, то при генерации его не создало, а также оно не будет храниться в нашей таблице заявок, то просто сделаем его свойством нашей заявки. Добавляем его в модель «Request.php»

```
24 class Request extends \yii\db\ActiveRecord
25 {
    no usages
26     public $other_request;
27 }
```

Не забываем это свойство написать в метод «attributeLabels», заодно можно сразу перевести все поля

```

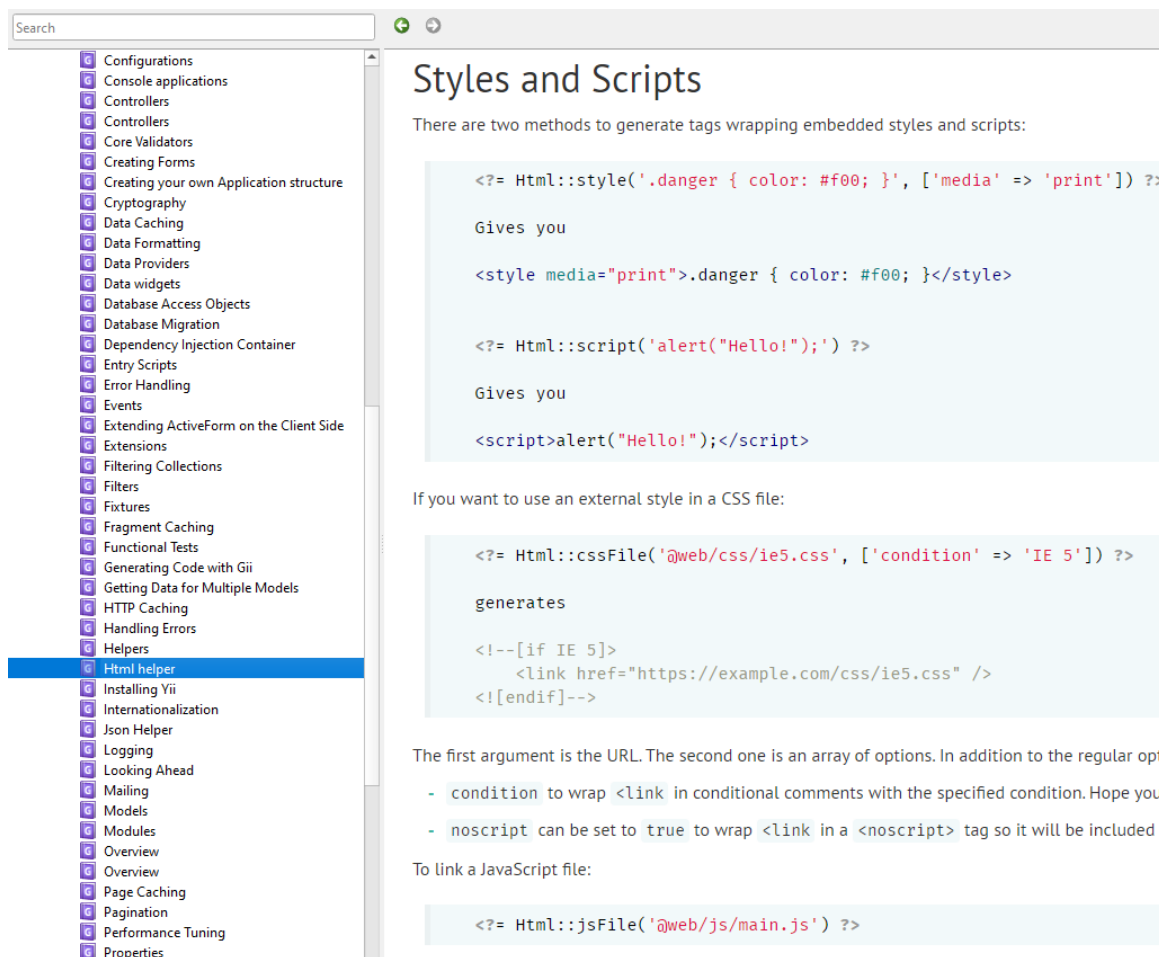
57  public function attributeLabels()
58  {
59      return [
60          'id' => 'ID',
61          'user_id' => 'Пользователь',
62          'service_id' => 'Вид услуги',
63          'address' => 'Адрес',
64          'contact_phone' => 'Номер телефона',
65          'desired_datetime' => 'Желаемое дата и время',
66          'custom_service_description' => 'Ваша услуга',
67          'payment_method' => 'Предпочтительный тип оплаты',
68          'status' => 'Статус',
69          'cancellation_reason' => 'Причина отмены',
70          'other_request' => 'Иная услуга'
71      ];
72  }

```

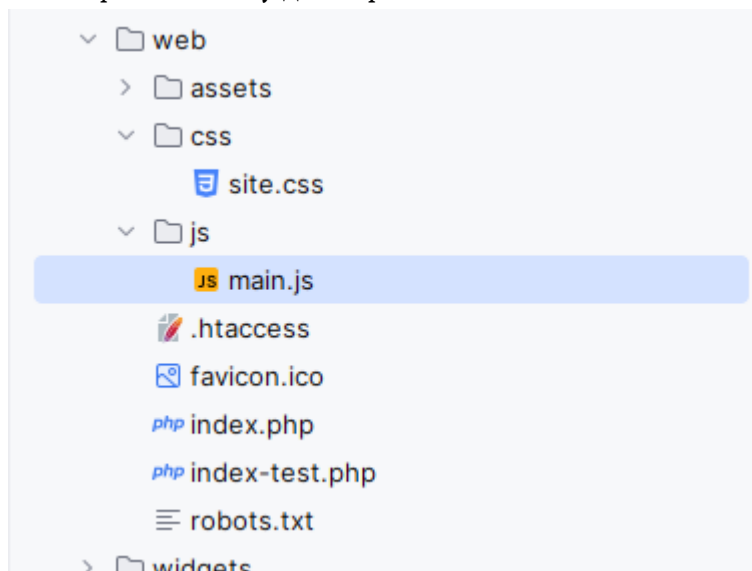
На этом моменте у нас **наступает развилка** в том, как мы будем реализовывать **отображение поля для «Иная услуга»**, поэтому тут будем рассматривать **два варианта**. **Первый вариант сложнее**, но более изящный, **второй проще** и в принципе позволяет выполнить задание, поэтому **если у вас не очень хорошо с JavaScript (jQuery)**, то сразу **листайте на вариант 2**.

Вариант 1 (JavaScript или jQuery).

Если вы пошли этим путём, то вы хотите сделать всё «красиво», поэтому так и поступим. **Мы могли бы скрипт написать сразу в форме, но так лучше не делать**, поэтому идём в нашу справку «**Zeal**» и находим раздел «**Html Helper**», там есть статья о том, как подключать внешние файлы



В скриншоте внизу как раз видно **пример подключения JavaScript файла**. Для начала создаём наш файл и папку для скрипта в «web»



Теперь нужно его подключить. Открываем наше основное представление «views/layouts/main.php» и добавляем наше подключение в заголовок, но не забываем, что нам **нужно дождаться загрузки DOM**, а потом выполнять скрипт, поэтому параметром добавим атрибут «defer»

```

24 <html lang="<?= Yii::$app->language ?>" class="h-100">
25 <head>
26     <title><?= Html::encode($this->title) ?></title>
27     <?php $this->head() ?>
28     <?= Html::jsFile(url: '@web/js/main.js', ['defer' => true]) ?>
29 </head>

```

Совет! Проверьте что всё работает, напечатав alert в файл «main.js»

Логика скрипта будет простой. Сначала выбираем каждый из наших 3-х элементов (чекбокс, блок с полем выбора сервиса, блок с полем ввода для иной услуги). Отключаем отображение блок для собственной услуги. А потом вешаем прослушку события на наш чекбокс на его изменение и просто меняем отображение блоков.

```

1  const check = $("input[id*='other_request']");
2  const serviceField = $("div[class*='service_id']");
3  const customServiceField = $("div[class*='custom_service_description']");
   Show usages
4  function toggleFields() {
5      if (check.is(selector: ':checked')) {
6          serviceField.show().toggle('fade');
7          customServiceField.hide().toggle('fade');
8      } else {
9          serviceField.hide().toggle('fade');
10         customServiceField.show().toggle('fade');
11     }
12 }
13 toggleFields();
14 check.on('change', toggleFields);

```

Здесь у нас должно начать работать переключение, но мы ещё можем улучшить наше переключение. Так как у нас есть «jQuery» и «Bootstrap», то код можно написать так, чтобы ещё была анимация переключения:

Вариант 2 (без JavaScript)

Тут важно понять, что у нас будут отображаться оба поля заявок и галочка, поэтому обязательность выбора одного из вариантов мы должны привязать к нашему чекбоксу.

Для начала удаляем поля для статуса и причины отмены, так как они нам нужны для администратора, а не для пользователя.

То есть форма для заявки пользователя у нас должна иметь такой вид (не важно есть ли JavaScript)

```
11 <div class="request-form">
12
13     <?php $form = ActiveForm::begin();
14     $services = \app\models\Service::find()
15         ->select(['name'])
16         ->orderBy( column: 'id')
17         ->column();
18     ?>
19
20     <?= $form->field($model, attribute: 'address')->textInput(['maxlength' => true]) ?>
21
22     <?= $form->field($model, attribute: 'contact_phone')->widget( class: \yii\widgets\MaskedInput::class, [
23         'mask' => '+7(999)-999-99-99', 'options' => ['value' => Yii::$app->user->identity->phone]
24     ]) ?>
25
26     <?= $form->field($model, attribute: 'desired_datetime')->input( type: 'datetime-local') ?>
27
28     <?= $form->field($model, attribute: 'service_id')->dropDownList($services) ?>
29
30     <?= $form->field($model, attribute: 'other_request')->checkbox() ?>
31
32     <?= $form->field($model, attribute: 'custom_service_description')->textInput(['maxlength' => true]) ?>
33
34     <?= $form->field($model, attribute: 'payment_method')->
35     dropDownList(['cash' => 'наличные', 'card' => 'банковская карта']) ?>
36
37     <div class="form-group">
38         <?= Html::submitButton( content: 'Отправить заявку', ['class' => 'btn btn-success']) ?>
39     </div>
```

Обратите внимание, что в скриншоте переведены поле «Предпочтительный тип оплаты» и кнопка отправки данных.

Теперь добавим правила валидации для нашей модели «Request.php». Два поля мы должны заполнить через код, поэтому зададим им значения по умолчанию (для пользователя и статуса)

```
53         ['user_id', 'default', 'value' => Yii::$app->user->identity->id],
54         ['status', 'default', 'value' => 'new'],
```

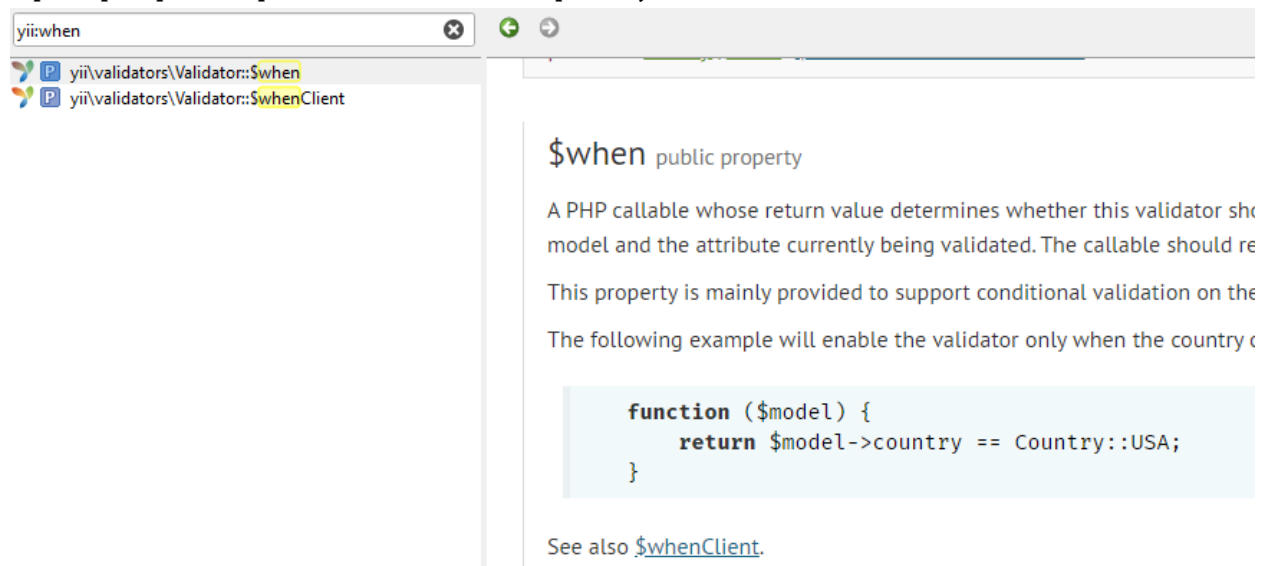
Также мы могли перенести сюда валидацию для номера телефона (по необходимости)

```
55         ['contact_phone', 'match', 'pattern' => '/^\\+?7\\(\\d{3}\\)-\\d{3}-\\d{2}-\\d{2}$/',
```

Ещё нам нужно добавить одно правило для нашего чекбокса собственной услуги, чтобы сработало правило очищения основной услуги

```
56         ['other_request', 'safe'],
```

Следующую правило валидации нам необходимо для обязательности заполнения поля текста, если была выбрана «иная услуга». Для начала в поиске «Zeal» найдём два правила «**when**», которое будет нам проверять чекбокс и включать правило валидации. Пример первого правила можно посмотреть тут:



The screenshot shows a search result for 'yii:when'. The left sidebar lists two items: 'yii\validators\Validator::\$when' and 'yii\validators\Validator::\$whenClient'. The main panel displays the definition for '\$when' as a public property. It explains that it is a PHP callable whose return value determines whether the validator should validate the model and the attribute currently being validated. It also mentions that this property is mainly provided to support conditional validation on the country attribute. A code example is provided:

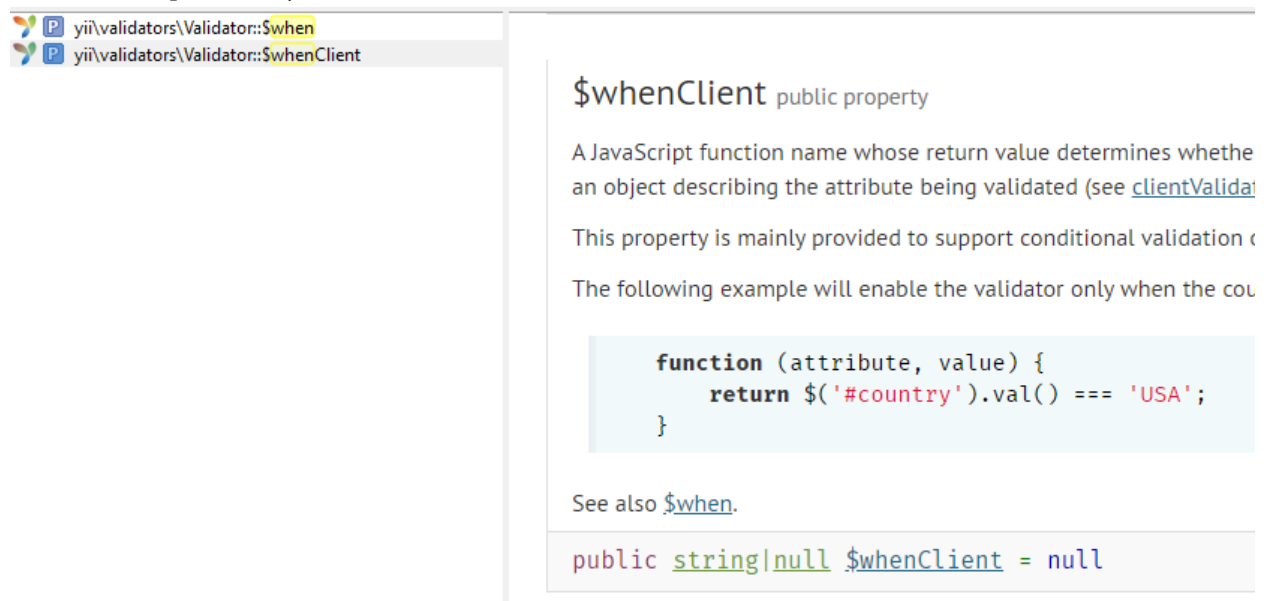
```
function ($model) {
    return $model->country == Country::USA;
}
```

Below the code example, it says "See also [\\$whenClient](#)."

Но в нашем случае из модели мы должны брать нашу «галочку» в «**other_request**», поэтому правило будет таким:

```
'when' => function ($model) {
    return $model->other_request;
},
```

Теперь нам нужно «**whenClient**»



The screenshot shows a search result for 'yii:whenClient'. The left sidebar lists two items: 'yii\validators\Validator::\$when' and 'yii\validators\Validator::\$whenClient'. The main panel displays the definition for '\$whenClient' as a public property. It explains that it is a JavaScript function name whose return value determines whether an object describing the attribute being validated (see [clientValidation](#)). It also mentions that this property is mainly provided to support conditional validation on the country attribute. A code example is provided:

```
function (attribute, value) {
    return $('#country').val() === 'USA';
}
```

Below the code example, it says "See also [\\$when](#)."

At the bottom of the panel, there is a code snippet:

```
public string|null $whenClient = null
```

Опять же тут нам нужно переписать пример в такой вид (id можно посмотреть через инспектор в браузере):

```

'whenClient' => "function(attribute, value) {
    return $('#request-other_request').is(':checked');
}";

```

Ещё мы можем сделать сообщение на случай ошибки валидации и получить в полном объёме такое правило:

```

57         ['custom_service_description', 'required',
58         'when' => function ($model) {
59             return $model->other_request;
60         },
61         'whenClient' => "function(attribute, value) {
62             return $('#request-other_request').is(':checked');
63         }";
64         'message' => 'Заполните описание услуги'
65     ],

```

Почти все правила валидации для нашей модели были сгенерированы автоматически, но в целом у нас должно быть так:

```

40  public function rules()
41  {
42      return [
43          [['address', 'contact_phone', 'desired_datetime', 'payment_method'], 'required'],
44          [['user_id', 'service_id'], 'integer'],
45          [['desired_datetime'], 'safe'],
46          [['payment_method', 'status'], 'string'],
47          [['address', 'contact_phone', 'custom_service_description', 'cancellation_reason'],
48              'string', 'max' => 255],
49          [['service_id'], 'exist', 'skipOnError' => true, 'targetClass' => Service::class,
50              'targetAttribute' => ['service_id' => 'id']],
51          [['user_id'], 'exist', 'skipOnError' => true, 'targetClass' => User::class,
52              'targetAttribute' => ['user_id' => 'id']],
53          [['user_id', 'default', 'value' => Yii::$app->user->identity->id],
54          ['status', 'default', 'value' => 'new'],
55          ['contact_phone', 'match', 'pattern' => '/^+?7(\d{3})-\d{3}-\d{2}-\d{2}$/',
56          ['other_request', 'safe'],
57          ['custom_service_description', 'required',
58              'when' => function ($model) {
59                  return $model->other_request;
60              },
61              'whenClient' => "function(attribute, value) {
62                  return $('#request-other_request').is(':checked');
63              }";
64              'message' => 'Заполните описание услуги'
65          ],
66      ];
67  }

```


Осталось только добавить правило проверки выбора «иная услуга». Будем очищать поле с id основной услуги, если пользователь выбрал свою услугу. Для этого допишем метод «beforeSave» в нашу модель «Request» и там напишем очистку данных

```
109  public function beforeSave($insert)
110  {
111      if ($this->other_request) {
112          $this->service_id = null;
113      }
114      return parent::beforeSave($insert);
115  }
```

Здесь у нас уже точно должна работать подача заявки со всеми валидациями.

Можно сделать ещё переадресацию после подачи заявки на все заявки

```
75  public function actionCreate()
76  {
77      $model = new Request();
78
79      if ($this->request->isPost) {
80          if ($model->load($this->request->post()) && $model->save()) {
81              return $this->redirect(['request/index']);
82          }
83      } else {
84          $model->loadDefaultValues();
85      }
```

Формирование ответов на заявки администратором

Для реализации функций администратора лучше делать отдельный контроллер, модель и представления. Но для решения задания достаточно **визуально разделить интерфейс** нашего приложения (мы уже так делали с кнопкой подачи заявки). То изменим теперь отображение «**views/request/index.php**»

```
19 <?php if (!Yii::$app->user->isGuest && !Yii::$app->user->identity->isAdmin()) {
20     echo Html::a( text: 'Подать заявку', ['create'], ['class' => 'btn btn-success']);
21     echo GridView::widget([
22         'dataProvider' => $dataProvider,
23         'columns' => [
24             ['class' => 'yii\grid\SerialColumn'],
25
26             'id',
27             'user_id',
28             'service_id',
29             'address',
30             'contact_phone',
31             //'desired_datetime',
32             //'custom_service_description',
33             //'payment_method',
34             //'status',
35             //'cancellation_reason',
36             [
37                 'class' => ActionColumn::className(),
38                 'urlCreator' => function ($action, Request $model, $key, $index, $column) {
39                     return Url::toRoute([$action, 'id' => $model->id]);
40                 },
41             ],
42         ],
43     ],
44 );
45 } else if (!Yii::$app->user->isGuest && Yii::$app->user->identity->isAdmin()) {
46     echo GridView::widget([
```

Теперь если у нас пользователь видит свой интерфейс. А вот администратор должен видеть только свои функции, по сути, это будет «**update**». Сделаем это в виде кнопки «**Ответить на заявку**»

Мы пока не касаемся какие поля отображать в списке заявок, так как это не сказано явным образом в задании и реализуем это чуть позже.

```

45 } else if(!Yii::$app->user->isGuest && Yii::$app->user->identity->isAdmin()) {
46     echo GridView::widget([
47         'dataProvider' => $dataProvider,
48         'columns' => [
49             ['class' => 'yii\grid\SerialColumn'],
50
51             'id',
52             'user_id',
53             'service_id',
54             'address',
55             'contact_phone',
56             //'desired_datetime',
57             //'custom_service_description',
58             //'payment_method',
59             //'status',
60             //'cancellation_reason',
61             [
62                 'class' => ActionColumn::className(),
63                 'template' => '{update}',
64                 'urlCreator' => function ($action, Request $model, $key, $index, $column) {
65                     return Url::toRoute([$action, 'id' => $model->id]);
66                 },
67                 'buttons' => [
68                     'update' => function ($url) {
69                         return Html::a(
70                             text: '<span class="btn btn-primary" type="button">ответить на заявку</span>', $url);
71                     },
72                 ],
73             ],
74         ]);
75 } ?>

```

Мы создавали форму для администратора, которую дублировали из основной формы, теперь сделаем на неё ссылку при вызове нашего обновления в «**views/request/update.php**» (лишние элементы «хлебных крошек» можете удалить)

```

8 $this->title = 'Ответ на заявку пользователя: ' . Html::encode($model->user->full_name);
9 // "хлебные крошки" удалены
10 ?>
11 <div class="request-update">
12
13     <h1><?= Html::encode($this->title) ?></h1>
14
15     <?= $this->render( view: '_formAdmin', [
16         'model' => $model,
17     ]) ?>
18
19 </div>

```

Мы можем вывести данные о заявке пользователя в форме (хотя это явно по заданию не написано), переходим в нашу форму админа «**views/request/_formAdmin.php**». Вы можете выводить поля на своё усмотрение.

```

13     <?php $form = ActiveForm::begin();
14     $request = app\models\Request::findOne($model->id);
15     ?>
16
17     <h2>Услуга -
18         <?php
19             $service = app\models\Service::findOne($model->service_id);
20             echo $service ? Html::encode($service->name) :
21                 Html::encode($request->custom_service_description);
22             ?>
23             (Телефон: <?= Html::encode($request->contact_phone); ?>)
24     </h2>
25
26     <p>Адрес: <?= Html::encode($request->address); ?></p>

```

Время и способ оплаты оформим в таком виде:

```

26     <p>Адрес: <?= Html::encode($request->address); ?></p>
27
28     <h3><?= $model->getAttributeLabel( attribute: 'desired_datetime' ) ?></h3>
29     <p><?= Html::encode(date( format: 'd-m-Y H:m', strtotime($request->desired_datetime))) ?></p>
30
31     <h3><?= $model->getAttributeLabel( attribute: 'payment_method' ) ?></h3>
32     <p><?= ($request->payment_method == "card") ? "банковская карта" : "наличные"; ?></p>
33

```

А вот поле отмены заявки должно быть связано с полем статуса. Поэтому нам понадобится их связать и отображать причину, если заявка будет отменена. Поля оформляем почти в исходном виде, убрав только «Новая» (сразу ещё переводим кнопку ответа)

```

34     <?= $form->field($model, attribute: 'status')->dropDownList([ 'in_progress' => 'В работе',
35         'completed' => 'Выполнено', 'cancelled' => 'Отменено' ], ['prompt' => '']) ?>
36
37     <?= $form->field($model, attribute: 'cancellation_reason')->textInput(['maxlength' => true]) ?>
38
39     <div class="form-group">
40         <?= Html::submitButton( content: 'Ответить', ['class' => 'btn btn-success']) ?>
41     </div>

```

Опишем переключение поля «Причина отмены», когда выбрано «Отменено» через скрипт «web/js/main.js» (Внимание! Если вы пропустили этот шаг, то сейчас тоже можете его не делать, функционал не будет потерян)

```

16     const status = $("select[id*='request-status']");
17
18     Show usages
19     function toggleCancellationReason() {
20         const customField = $("div[class*='cancellation_reason']");
21         if(status.val() !== 'cancelled') customField.show().toggle('fade');
22         else customField.hide().toggle('fade');
23     }
24     toggleCancellationReason();
25     status.on('change', toggleCancellationReason);

```

Так как у нас оба обращения (заявка и ответ) происходят через одну модель, то нам необходимо обернуть условия валидации в зависимость от роли, поэтому для пользователя добавим условие

```

40  public function rules()
41  {
42      if (!Yii::$app->user->identity->isAdmin()) {
43          return [
44              [['address', 'contact_phone', 'desired_datetime'
45              [['user_id', 'service_id'], 'integer'],

```

А на «else» мы должны написать правила: обязательность для статуса и обязательность заполнения поля «Причины отмены», если отменили заявку (подобное уже проделывали ранее)

```

65          'message' => 'Заполните описание услуги'
66      ],
67  ];
68  } else {
69      return [
70          [['status'], 'required'],
71          ['cancellation_reason', 'required',
72              'when' => function ($model) {
73                  return $model->status == 'cancelled';
74              },
75              'whenClient' => "function(attribute, value) {
76                  return $('select[id*='request-status']').val() === 'cancelled';
77              }",
78              'message' => 'Заполните причину отмены'
79          ],
80      ];
81  }
82  }

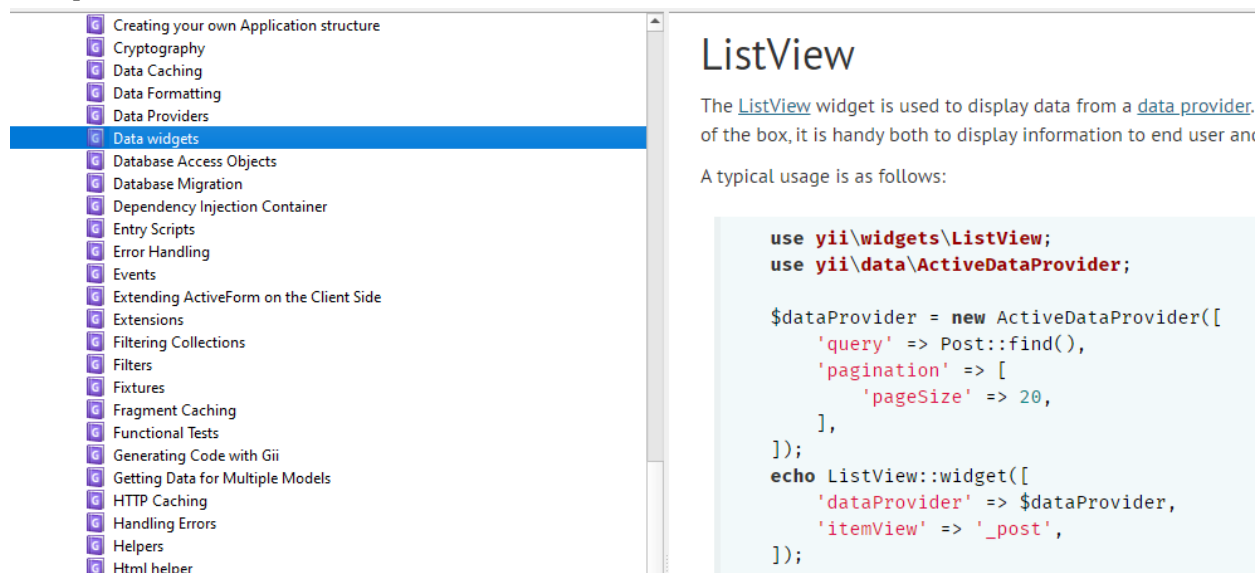
```

Принцип работы сайта в целом готов, но в следующей главе мы рассмотрим примеры как оформить сайт.

Дополнительное улучшение интерфейсов

Пользовательские карточки

Для начала нам необходимо разобраться в инструменте, который мы будем использовать для отображения. Первым делом поменяем виджет, который нам позволит отобразить всё не в табличных данных «**ListView**»



The screenshot shows the Yii2 documentation page for the **ListView** widget. On the left is a sidebar with a list of topics, where 'Data widgets' is selected. The main content area has the title 'ListView' and a description: 'The **ListView** widget is used to display data from a [data provider](#). of the box, it is handy both to display information to end user and...'. Below this, it says 'A typical usage is as follows:' and provides a code snippet:

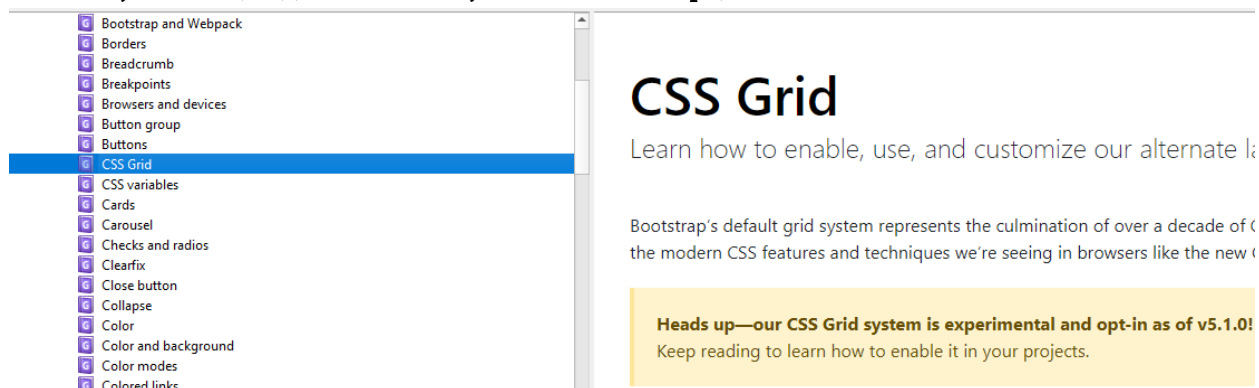
```
use yii\widgets\ListView;
use yii\data\ActiveDataProvider;

$dataProvider = new ActiveDataProvider([
    'query' => Post::find(),
    'pagination' => [
        'pageSize' => 20,
    ],
]);
echo ListView::widget([
    'dataProvider' => $dataProvider,
    'itemView' => '_post',
]);
```

На основе справки мы можем **заменить** наш «**GridView**» на «**ListView**», а также давайте **изменим условную отрисовку** в нашем файле «**views/request/index.php**» (после этого вы можете увидеть на странице индексы из БД, а это значит что вы делаете всё правильно)

```
20 <?php if (!Yii::$app->user->isGuest && !Yii::$app->user->identity->isAdmin()) {
21     echo Html::a( text: 'Подать заявку', ['create'], ['class' => 'btn btn-success']);
22     echo ListView::widget([
23         'dataProvider' => $dataProvider,
24     ]);
```

Визуальное оформление можно организовать через встроенный «**Bootstrap5**», для этого воспользуемся ещё одной статьей уже из «**Bootstrap5/Guides**»

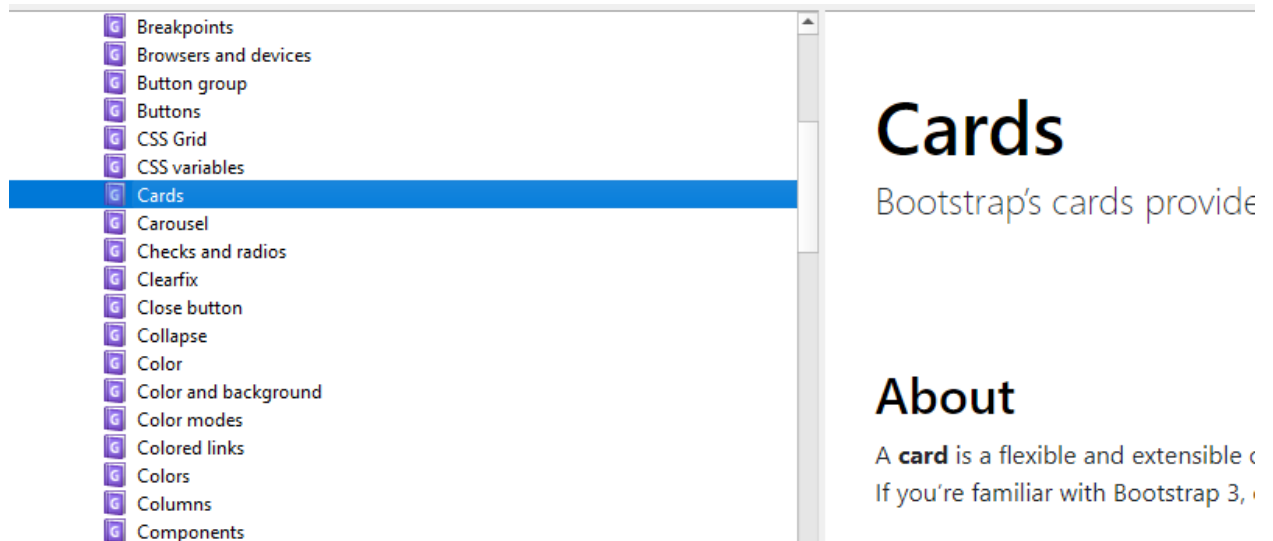


The screenshot shows the Bootstrap5 documentation page for the **CSS Grid** system. On the left is a sidebar with a list of topics, where 'CSS Grid' is selected. The main content area has the title 'CSS Grid' and a subtitle 'Learn how to enable, use, and customize our alternate layout system'. Below this, it says 'Bootstrap's default grid system represents the culmination of over a decade of CSS grid features and techniques we're seeing in browsers like the new CSS Grid'. At the bottom, there is a yellow box with the text: 'Heads up—our CSS Grid system is experimental and opt-in as of v5.1.0! Keep reading to learn how to enable it in your projects.'

Добавим наши **параметры колонок** для будущих карточек заявок через параметры

```
22         echo ListView::widget([
23             'dataProvider' => $dataProvider,
24             'options' => ['class' => 'row g-4'],
25             'itemOptions' => ['class' => 'col-md-4'],
26         ]);
```

Само **оформление карточек** мы опять же возьмём из статьи, где показаны примеры



Все карточки будем писать через параметр «**itemView**», возвращая всё в виде строки (параметры «**\$index**» и «**\$widget**» мы можем удалить, так как не будем использовать)

```
25         'itemOptions' => ['class' => 'col-md-4'],
26         'itemView' => function ($model, $key, $index, $widget) {
27             return '';
28         }
```

Создадим теперь карточку и в заголовок карточки добавим номер заявки нашего пользователя

```
26         'itemView' => function ($model, $key) {
27             return '
28             <div class="card shadow-sm">
29                 <div class="card-header bg-secondary text-white">
30                     <h5 class="card-title">Заявка № ' . Html::encode($key) . ' </h5>
31                 </div>
32             </div>
33             ';
```

Теперь выведем пользователю заказанную им услугу, но не забываем что у нас может быть не только услуга из списка, но и «своя», поэтому нам нужно собрать данные исходя из этого

```
27         'itemView' => function ($model, $key) {
28             $service_name = Service::findOne($model->service_id)->name
29             ?? $model->custom_service_description." (Своя услуга)";
30             return '

```

Теперь мы можем отобразить эти данные пользователю

```
31 <div class="card shadow-sm">
32   <div class="card-header bg-secondary text-white">
33     <h5 class="card-title">Заявка № ' . Html::encode($key)
34   </div>
35   <div class="card-body">
36     <p class="d-flex justify-content-between">
37       <b>Услуга: </b>'
38       . $service_name .
39     ' </p>
40   </div>
41 </div>
```

Остальные поля в задании явно не определены, поэтому просто сделаем их по аналогии

```
38   . $service_name .
39   ' </p>
40   <p class="d-flex justify-content-between">
41     <b>Адрес: </b>'
42     . $model->address .
43   ' </p>
44   <p class="d-flex justify-content-between">
45     <b>Телефон: </b>'
46     . $model->contact_phone .
47   ' </p>
48   <p class="d-flex justify-content-between">
49     <b>Желательное время: </b>'
50     . date( format: 'd-m-Y H:i', strtotime($model->desired_datetime)) .
51   ' </p>
52 </div>
```

Так как способ оплаты мы задали не через кириллицу, то напомним ассоциативный массив с ключами из нашей БД, а значение на русском

```
28 $service_name = Service::findOne($model->service_id)->name
29 ?? $model->custom_service_description." (Своя услуга)";
30 $payment = ['cash' => 'Наличные', 'card' => 'Банковская карта'];
31 return ' </div>
```

Из модели по ключу будем русифицировать отображение в таком виде:

```
51   . date( format: 'd-m-Y H:i', strtotime($mo
52   ' </p>
53   <p class="d-flex justify-content-between">
54     <b>Способ оплаты: </b>'
55     . $payment[$model->payment_method] .
56   ' </p>
57 </div>
```

Статус заявки мы отобразим в «подвале» карточки, но для начала по аналогии со способом оплаты заведём ассоциативный массив, но сделаем его сложнее (ниже рассмотрим почему так)


```

30     $payment = ['cash' => 'Наличные', 'card' => 'Банковская карта'];
31     $statuses = ["new" => ["name" => "Новая", "class" => "bg-secondary bg-gradient"],
32                 "in_progress" => ["name" => "В работе", "class" => "bg-primary bg-gradient"],
33                 "cancelled" => ["name" => "Отменено", "class" => "bg-danger bg-gradient"],
34                 "completed" => ["name" => "Выполнено", "class" => "bg-success bg-gradient"]
35     ];
36     return '

```

Ещё определим **не было ли отмены заявки** (необходимо отобразить причину)

```

34         "completed" => ["name" => "Выполнено", "class" => "bg-success bg-gradient"]
35     ];
36     $current_status = $model->status == "cancelled"
37         ? $statuses[$model->status]["name"] . ' (' . $model->cancellation_reason . ')'
38         : $statuses[$model->status]["name"];
39     return '

```

Отображение будет похоже, но тут мы применим класс для «подвала» карточки (смотрим в справку)

```

63         . $payment[$model->payment_method] .
64         '</p>'
65     </div>
66     <div class="card-footer d-flex justify-content-between">
67         <b>Статус: </b>'
68         . $current_status .
69     </div>
70 </div>
71 ';

```

Используем второй ключ «class» из нашего массива «\$statuses» для визуального отображения разной заливки «шапки» карточки в зависимости от статуса

```

40     <div class="card shadow-sm">
41         <div class="card-header ' . $statuses[$model->status]["class"] . ' text-white">
42             <h5 class="card-title">Заявка № ' . Html::encode($key) . '</h5>
43         </div>

```

Ещё можно добавить пользователю возможность удалять не обработанные (новые) заявки. Определим, как будет выглядеть этот элемент в таком виде:

```

38         : $statuses[$model->status]["name"];
39     $delete = ($model->status === "new") ? Html::a(
40         'text: '&#x2620;', ['delete', 'id' => $model->id], [
41         'class' => 'btn btn-danger p-1 text-white',
42         'data-confirm' => 'Вы уверены, что хотите удалить этот элемент?']) : "";
43     return '

```

И добавим нашу кнопку в шаблон

```

43     <div class="card shadow-sm">
44         <div class="card-header '
45         . $statuses[$model->status]["class"] .
46         ' text-white d-flex justify-content-between">
47             <h5 class="card-title">Заявка № ' . Html::encode($key) . '</h5>
48             ' . $delete . '
49         </div>

```

Для отрисовки карточек администратору мы используем этот же шаблон, только заменим возможность удаления карточки на ответ на заявку, для этого опишем условия работы этой кнопки (только если статус «Новая» или «В работе»)

```

94     $current_status = $model->status == "cancelled"
95     ? $statuses[$model->status]["name"] . ' (' . $model->cancellation_reason . ')'
96     : $statuses[$model->status]["name"];
97     $answer = ($model->status === "new" || $model->status === "in_progress") ?
98     Html::a( text: '#128584;', ['update', 'id' => $model->id], [
99     'class' => 'btn btn-dark p-1 text-white']) : "";
100     return '
101     <div class="card shadow-sm">

```

И теперь выведем эту кнопку для нашего администратора

```

102     <div class="card-header "
103     . $statuses[$model->status]["class"] .
104     ' text-white d-flex justify-content-between">
105     <h5 class="card-title">Заявка № ' . Html::encode($key) . ' </h5>
106     | . $answer . '
107     </div>

```

В задании не указано какие именно данные должен видеть администратор, поэтому мы в карточке выводим всю информацию. В случае администратора нам не хватает только вывести ФИО клиента

```

106     | . $answer . '
107     </div>
108     <div class="card-body">
109     <p class="d-flex justify-content-between">
110     <b>ФИО: </b>'
111     . app\models\User::findOne($model->user_id)->full_name .
112     '</p>
113     <p class="d-flex justify-content-between">
114     <b>Услуга: </b>'
115     . $service_name .
116     '</p>

```

Можно ещё сделать переадресацию после смены статуса в нашем контроллере «RequestController.php»

```

99     public function actionUpdate($id)
100     {
101         $model = $this->findModel($id);
102
103         if ($this->request->isPost && $model->load($this->request->post()) && $model->save()) {
104             return $this->redirect(['request/index']);
105         }
106
107         return $this->render( view: 'update', [
108             'model' => $model,
109         ]);
110     }

```

Задание полностью сделано. Остальные улучшения вы можете произвести самостоятельно.

От автора. В методичке рассмотрен вариант отрисовки через представление, но в реальности так нельзя делать, но задача сводилась только к тому, чтобы выполнить задание самым простым и быстрым способом. Здесь только показан один из вариантов решения. Спасибо за внимание и желаю вам удачи на экзамене!